

SPECK: Signatures from Permutation Equivalence of Codes and Kernels

Marco Baldi¹, Michele Battagliola¹, Rahmi El Mechri¹,
Paolo Santini¹, Riccardo Schiavoni¹, and Davide De Zuane^{1,2}

¹Marche Polytechnic University, Ancona, Italy

² IMT School for Advanced Studies, Lucca, Italy

Abstract. The ongoing search for secure and efficient post-quantum cryptographic primitives has led to the development of numerous novel digital signature schemes. In this paper we introduce **SPECK**, a new code-based digital signature scheme exploiting the similarities between the Permuted Kernel Problem (PKP) and the Permutation Code Equivalence Problem (PEP). At its core, **SPECK** is based on the permutation version of LESS, but introduces a key modification to the commitment step. Indeed, instead of committing to an entire permuted code, in **SPECK** the prover commits to a random relaxed PKP (that we call **PECK**, Permutation Equivalence of Codes and Kernel) instance by randomly choosing a codeword from a random permutation of the initial code. In this sense, the secret key is used as a trapdoor to solve the committed **PECK** instance. The new approach allows us to pick the best from both worlds: it achieves fast verification time, since it only needs to perform some vector-matrix multiplication, thus avoiding the expensive Gaussian elimination required in LESS. Concurrently, by using PKP solely as a trapdoor for LEP, we maintain a simple three-round protocol, circumventing the need for more complex techniques like MPC-in-the-Head or multiple rounds, resulting in a signature size that remains comparable to LESS and significantly shorter than PERK.

Keywords: Digital Signatures, Code-based cryptography, Code Equivalence, PKP, Zero-Knowledge Proofs

1 Introduction

In June 2023, NIST issued an additional call for post-quantum signature schemes, specifically asking for algorithms with short signatures, fast verification and no security assumption from structured lattices.¹ In the first round, 40 algorithms have been accepted as complete and proper; in October 2024, NIST has announced the 14 algorithms that have been promoted to the second round [3].

One of these schemes is LESS, which is obtained by converting an interactive Zero Knowledge (ZK) proof for the code equivalence problem into a digital signature via the Fiat-Shamir transform [15]. In the case of LESS, codes are endowed

¹ <https://csrc.nist.gov/projects/pqc-dig-sig>

with the Hamming metric, hence an equivalence is nothing but a *monomial map*, that is, a linear transformation that scales and shuffles coordinates.² The problem of finding a monomial transformation that maps a code onto another code is known as Linear Equivalence Problem (LEP).

In LESS, one wants to prove knowledge of a monomial transformation μ (being the user's private key) which maps one public code onto another public code (being the user's public key). Since codes are customary represented via their generator matrices, the public key is a pair of matrices $\mathbf{G}, \mathbf{G}' = \mathbf{S} \cdot \mu(\mathbf{G}) \in \mathbb{F}_q^{k \times n}$, where $\mathbf{G}$ is a generator matrix for the public code $\mathcal{C}$, $\mathbf{G}'$ is a generator matrix for the permuted code $\mathcal{C}'$ and $\mathbf{S}$ is a $k \times k$ non-singular matrix. In each round of the ZK protocol, the prover samples a random monomial transformation $\tau \xleftarrow{\$} M_n$ and commits to $\mathbf{SF}(\tau(\mathbf{G}))$, where $\mathbf{SF}$ denotes the function that returns the generator matrix in systematic form. Later on, as responses, the prover either discloses τ or $\tau' = \tau \circ \mu^{-1}$: in the former case, the verifier just repeats the very same operations performed by the prover, while in the latter case they check that the commitment can be obtained as $\mathbf{SF}(\tau'(\mathbf{G}'))$.

Observe that the responses in the protocol are only monomial transformations and not changes of basis. This is made possible since the systematic form is an invariant under changes of basis. On one hand, this reduces significantly the communication cost, since it allows to avoid communicating the basis change; on the other hand, computing the systematic form takes time $O(n^3)$, causing a major slow down in the signature computation.

Since its initial proposal in [11], the LESS scheme has undergone significant improvements such as the usage of multi-keys, fixed-weight challenges and canonical forms [7, 14, 20], that are all used in the final NIST submission. While these optimizations help in compressing the signature size, they all still require the computation of the systematic form, that is still the main bottleneck in terms of computational cost.

1.1 Our contribution

In this paper, we propose a scheme which is inspired from LESS and aims to reduce its computational overhead. In a nutshell, instead of verifying the existence of a map that transforms the first code onto the second, we verify this relation only on some random codewords.

Using this idea, one can devise a simple ZK proof as in Figure 1. In particular, the secret key is a permutation $\pi \in S_n$, while the public key is a pair of codes $\mathcal{C}, \mathcal{C}' = \pi(\mathcal{C})$. The prover first samples a random isometry τ and then samples and commits to a random codeword $\mathbf{c} \xleftarrow{\$} \tau(\mathbf{c})$.

As responses, the prover shows either that the commitment has been honestly prepared, or that they know an isometry $\tau' = \pi \circ \tau^{-1}$ sending $\mathbf{c}$ to $\mathcal{C}'$.

² The notion of equivalence can be slightly generalised by considering also a field automorphism. In this paper, we are not concerned with this notion; in any case, one can even show that the hardness of the problem would not change significantly.

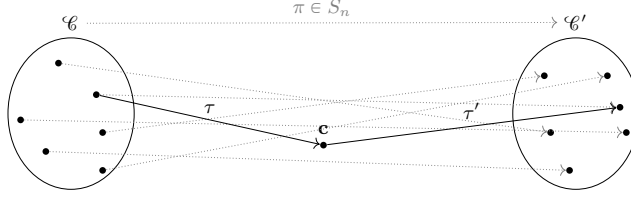


Fig. 1: Representation of the relations between $\mathcal{C}$, $\mathcal{C}'$, the secret π , the commitment $\mathbf{c}$ and the responses τ and τ' .

It is easy to see that all required operations, for both the prover and the verifier, boil down to some vector-matrix multiplications, which can be done in time $O(n^2)$, making it much faster than LESS. On the other hand, our protocol still maintains the classical and rather simple structure of a three-move protocol, without relying on more complex techniques such as MPCitH, TCitH or VOLEitH. This allows us to keep the signature size under control, since we only need three rounds instead of five or seven: compared to PERK, which also relies on PKP, SPECK achieves roughly half the signature size when using parameters that yield the same verification speeds.

Using permutations. Intuitively, forging signatures in SPECK boils down to finding a transformation sending $\mathbf{c}$ to $\mathcal{C}'$. If we allow the transformation to be any monomial one, then this is rather easy since it would be enough to find a codeword $\mathbf{c}'$ of $\mathcal{C}'$ having the same Hamming weight as $\mathbf{c}$: then, to find the monomial matrix, it is enough to consider a permutation mapping the non-zero elements of $\mathbf{c}$ into the non-zero elements of $\mathbf{c}'$ and adjust the scalar accordingly. This is the reason why we need to restrict ourselves to the use of permutations. Consequently, the security of our scheme depends on the Permutation Equivalence Problem (PEP), which is a special case of LEP. To avoid attacks based on the hull [6, 23], we need to use self-dual codes.

The PECK hardness assumption. In SPECK, forging signatures requires to efficiently find a permutation sending a random codeword $\mathbf{c}$ coming from $\mathcal{C}$ into $\mathcal{C}'$. We refer to this task as the Permuted Codes and Kernel (PECK) problem and accurately study its hardness. In some sense, this problem resembles the Permuted Kernel Problem (PKP) and, obviously, it is not harder than PEP and PKP. However, when $q = O\left((n/e)^{\frac{1}{1-k/n}}\right)$, which (heuristically) implies that there exists a unique permutation mapping $\mathbf{c}$ into $\mathcal{C}'$, we can show that the problem is expected to be as hard as PEP. We call this regime “*High q*”. We also study the “*Low q*” regime, in which an exponentially large number of solutions may exist. We accurately study how solvers for PKP can be adapted to this case. Our assessment requires some non trivial combinatorial arguments since, differently from the standard regime in which PKP is studied, here we have

multiple solutions and allow for repeated entries in $\mathbf{c}$. Still, we are able to show that the binary complexity of PKP-inspired attacks is much greater than the security parameter.

The resulting scheme. Starting from the simple scheme depicted in Figure 1, we optimise the protocol and reduce its communication cost from $n(\log_2(n) + \log_2(q))$ bits (the amount of bits necessary to represent a length- n vector and a permutation) to only $k \log_2(q)$ bits. We also rely on standard optimizations and, thanks to the Fiat-Shamir transform, obtain the signature scheme which we call **SPECK**.

We propose several instances of **SPECK**, aiming to achieve various trade-offs. The instances in the High q regime are more conservative, since they have a security which reduces to PEP and enjoy special soundness (under the heuristic assumption that there is a unique permutation sending $\mathbf{c}$ into $\mathcal{C}'$). However, they have larger signatures and are expected to be slower because of the large finite field size. The instances in the Low q regime have the same code parameters as the current **LESS** scheme. They are based on a less conventional approach and require an ad-hoc analysis, but enjoy shorter signatures and are much faster than **LESS**.

To verify the computational advantages of **SPECK**, we developed an optimized implementation, derived from the implementation of **LESS**³. In Figure 2 we compare the recommended instances of **SPECK** with the various Fiat-Shamir-based signatures from the second Round of the NIST call for additional signature schemes. Timings for these algorithms have been collected by performing benchmarks on an Intel(R) Core(TM) i5-10310U CPU clocked at 1.7GHz with Turbo Boost disabled. We only report results for the **SPECK** instances in the Low q regime. As it can be seen, our protocol achieves a good balance between signature size and verification time, achieving performance close to the best MPCitH protocols. In particular, when compared to the most similar signatures, namely **LESS** and **PERK**, **SPECK** stands out by being both shorter and faster than **PERK** and by being between 50 and 250 faster than **LESS**, at the price of only a 2 – 4 times bigger signature size. The code we used for the paper (including **SPECK** implementation and algorithms used for parameters estimation), can be found at <https://github.com/SPECK-Signature/SPECK>.

1.2 Paper organization

In Section 2 we introduce the notation used in this work and the preliminaries needed. Section 3 and Section 4 represent the core of our work. First, in Section 3 we describe the new interactive protocol and prove its security, next, in Section 4 we describe the signature scheme obtained from it, the optimizations used and its performance. In Section 5 we examine the commonly used multi-key optimization and highlight some critical problems that make it unusable in our context. In

³ <https://github.com/less-sig/LESS>

Section 6 we analyse the hardness assumption introduced by this work and provide insights about possible attacks and the choice of parameters. Finally, in Section 7 we draw some conclusions and pinpoint possible areas for improvement and future works.

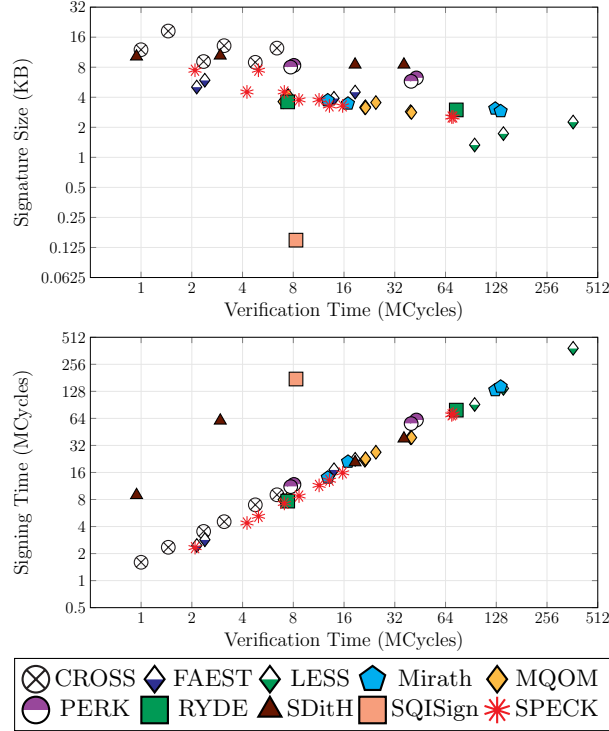


Fig. 2: Overview of the signature size and the verification key size of round 2 NIST additional signatures based on ZK, MPCitH and VOLE-in-the-Head frameworks. Benchmark was performed on an Intel(R) Core(TM) i5-10310U CPU clocked at 1.7 GHz with Turbo Boost disabled.

2 Preliminaries

In this section we establish the notation we use throughout the paper and recall the necessary background concepts about sigma protocols, signatures and linear codes.

2.1 Notation

In this paper, q denotes a prime and $\mathbb{F}_q$ indicates the finite field with q elements. We will use boldface uppercase letters to denote matrices, and boldface lowercase

letters to denote vectors. The identity matrix of size k is indicated as $\mathbf{I}_k$. The null vector or matrix is indicated as $\mathbf{0}$; the dimensions of $\mathbf{0}$ will always be clear from the context.

We denote by S_n the symmetric group on n elements and view its elements as permutations. We usually represent permutations through the associated $n \times n$ matrices and, with some abuse of notation, will write $\mathbf{P} \in S_n$ to specify that $\mathbf{P}$ is a permutation matrix. The group of non-singular square matrices with dimension k and values over $\mathbb{F}_q$ is denoted as $GL_k(\mathbb{F}_q)$.

Cryptographic notation As usual, λ indicates the security parameter. We use $\text{Hash} : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ to indicate a cryptographic hash function. We use the symbol $\xleftarrow{\$}$ to denote uniform sampling from a set. For a given distribution $\mathcal{D}$, we write $x \leftarrow \mathcal{D}$ if x is sampled from $\mathcal{D}$. Finally, if the seed **Seed** used for sampling is relevant we write $\xleftarrow{\text{Seed}}$.

When writing pseudocode we reserve the symbol $=$ for equality testing, while we use $x := a + b$ to denote that the variable x assume the value $a + b$.

Useful notions about digital signatures, sigma protocols and the Fiat-Shamir transform are given in Appendix A.

2.2 Linear Codes

A *linear code* $\mathcal{C}$ with length n and dimension k over $\mathbb{F}_q$ is a k -dimensional subspace of $\mathbb{F}_q^n$. An elements $\mathbf{c} \in \mathcal{C}$ is called codeword. We define *code rate* the quantity $R = k/n$. A code $\mathcal{C} \subseteq \mathbb{F}_q^n$ with dimension k can be represented through a *generator matrix*, that is, a full-rank $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ such that the rows form a basis for the code. Another representation can be obtained through a basis for the null space of the code, which is a subspace of $\mathbb{F}_q^n$ with dimension $n - k$ called *dual code* and indicated as $\mathcal{C}^\perp$. A basis for $\mathcal{C}^\perp$ is a full-rank matrix $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$, called *parity-check matrix*. For any generator matrix $\mathbf{G}$ and parity-check matrix $\mathbf{H}$ for the same code, it holds that $\mathbf{GH}^\top = \mathbf{0}$.

For any $\mathbf{S} \in GL_k(\mathbb{F}_q)$, $\mathbf{G}$ and $\mathbf{SG}$ generate the same code; the same applies to parity-check matrices, with changes of basis taken from $GL_{n-k}(\mathbb{F}_q)$. Among all possible representations, we sometimes desire to use matrices having a special form, called “systematic”. A systematic generator matrix is in the form $(\mathbf{I}_k, \mathbf{A})$ with $\mathbf{A} \in \mathbb{F}_q^{k \times (n-k)}$, from which we obtain the systematic parity-check matrix as $(\mathbf{B}, \mathbf{I}_{n-k})$, with $\mathbf{B} = -\mathbf{A}^\top$. We indicate as **SF** the function that, on input a generator matrix, returns the systematic form.

Notice that for some codes, the systematic form may not exist: this happens whenever the first k columns of a generator matrix do not form a non singular matrix (using the coding theory vocabulary, whenever the first k coordinates do not constitute an information set). In such cases, the notion of systematic form may be generalised through that of “standard” form, which allows for the $k \times k$ identity matrix to show up somewhere among the columns of $\mathbf{G}$; the same applies, with some evident adjustments, to parity-check matrices. In this paper, for the sake of simplicity, we will implicitly assume that all the matrices

we consider admit a systematic form. We point out that this simplification has no practical impact on our analysis, as everything we say can be generalised to matrices in standard form, and has the positive effect of unnecessary overloading the notation.

The hull of a linear code The hull of a code is the subspace given by the intersection between the code and its dual, i.e., $\mathcal{H}(\mathcal{C}) = \mathcal{C} \cap \mathcal{C}^\perp$. We say the hull of a code is trivial if contains only the null vector. As it is well known, random codes have trivial hull with large probability, approximately $1 - 1/q$ [24]. Typically, a code is called *weakly self-dual* if its hull has dimension $h > 1$, implying that it intersects non trivially (i.e., in a space of dimension ≥ 1) with its dual. When $h = k$, a code is called *self-orthogonal* and if the hull has maximum dimension $h = k = n/2$, we say the code is *self-dual*. An algorithm to sample codes having hull of a fixed dimension was recently proposed in [4]. A low hull dimension makes solving Permutation Equivalence easy [6, 23], for this reason throughout this paper we only consider self-dual codes.

2.3 PEP and PKP

In this paper, we consider the Permutation Equivalence Problem (PEP) which, given two linear codes, asks to find a permutation mapping a code onto the other. Formally, the problem reads as follows.

Problem 1 (Permutation Equivalence Problem (PEP)) *Given two linear codes $\mathcal{C}, \mathcal{C}' \subseteq \mathbb{F}_q^n$, with respective generator matrices $\mathbf{G}, \mathbf{G}' \in \mathbb{F}_q^{k \times n}$, find (if it exists) a permutation $\mathbf{P} \in S_n$ and a non singular $\mathbf{S} \in GL_k(\mathbb{F}_q)$ such that $\mathbf{G}' = \mathbf{SGP}$.*

A witness for a PEP instance can just be the permutation $\mathbf{P}$, as one can simply check that $\mathbf{SF}(\mathbf{GP}) = \mathbf{SF}(\mathbf{G}')$ or, equivalently, that $\mathbf{GPH}'^\top = \mathbf{0}$, with $\mathbf{H}'$ being a parity-check matrix for $\mathcal{C}'$.⁴

We further recall the Permuted Kernel Problem (PKP), describing it in terms of coding theory as this is convenient for our treatment.

Problem 2 (Permuted Kernel Problem (PKP)) *Given a linear code $\mathcal{C} \subseteq \mathbb{F}_q^n$ with parity-check matrix $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$ and a vector $\mathbf{c} \in \mathbb{F}_q^n$, find (if it exists) a permutation $\mathbf{P} \in S_n$ such that $\mathbf{cPH}^\top = \mathbf{0}$.*

A detailed discussion of the hardness of PEP and PKP can be found in [8, 22]. Here we summarise some general results:

⁴ This is an example of why the systematic form should, formally, be replaced with the standard form. Indeed, the verification based on the systematic forms would obviously fail when the first k columns of $\mathbf{G}'$ do not form a non singular matrix. Still, computing the standard form instead of the systematic form would do the job.

- PEP is easy when the hull is small, because of attacks such as [6, 23], whose complexity grows exponentially with the code dimension. These attacks are thwarted when one considers weakly self-dual codes with sufficiently large hull;
- all other attacks on PEP take time which is exponential in the code length. These type of attacks comprehend those based on low weight codewords [9, 18] and the meet-in-the-middle based on canonical forms [14];
- PKP has been normally studied in the scenario in which the input vector $\mathbf{c}$ has no repeated entry. This condition is easily satisfied when the finite field q is large enough. State-of-the-art attacks essentially require to brute-force over all permutations of $\mathbf{c}$, speeding up the procedure with standard techniques such as a meet-in-the-middle step and exploiting the parity-check equations described by $\mathbf{H}$.

As we shall see, forging signatures in our scheme, in the low q regime, will boil down to solving a PKP instance with, however, repeated entries. In Section 6 we recall in detail how PKP solvers work and describe how they adapt to our case.

2.4 Arrangements and Multiset

In this paper we often need “sets” having repeated entries and permutation on them. We refer to [19] and Appendix D for a detailed discussion and the proofs, here we summarize the most relevant notions.

Definition 1 (Multiset). *Given a set $\mathbb{S} = \{s_1, \dots, s_n\}$, a multiset $\mathbb{M}$ with basis $\mathbb{S}$ is defined as*

$$\mathbb{M} = \{s_1^{k_1}, \dots, s_n^{k_n}\}$$

where the indices k_i of elements s_i indicate the multiplicity of the occurrence of the element s_i in the multiset $\mathbb{M}$.

It is possible to extend the notion of permutation to multisets. In particular, we are interested in the notion of arrangements (often called partial permutation)

Definition 2 (Arrangement). *Given a multiset $\mathbb{M}$ of n different elements, and $k < n$ we define an arrangement over k (often called k -permutation) of a multiset $\mathbb{M}$ a sequence of k elements of $\mathbb{M}$ in which each element appears a number of times less than or equal to its multiplicity in $\mathbb{M}$.*

Let $\mathbb{X}$ be a set with $|\mathbb{X}| = q$ elements, and let $\mathbf{x} \in \mathbb{X}^n$ be a vector, without loss of generality, we identify the set $\mathbb{X}$ with the set $\{0, \dots, q-1\}$.

Let us define the mapping $\psi : \mathbb{X}^n \rightarrow \mathbb{N}^q$ such that:

$$\forall \mathbf{x} \in \mathbb{X}^n, \psi(\mathbf{x}) = (m_0, \dots, m_{q-1}) \in \mathbb{N}^q \quad (1)$$

Where i -th entry of $\psi(\mathbf{x})$ indicates the multiplicity of the value $i \in \mathbb{X}$ within $\mathbf{x}$. Through this mapping we can extend the definition of arrangements, as given for multisets, to vectors of length n and values in $\mathbb{X}$.

In particular, for $\mathbf{x} \in \mathbb{X}^n$ with $|\mathbb{X}| = q \leq n$ and a non-zero integer $t \leq n$, the set containing the arrangements of $\mathbf{x}$ on t entries is given by:

$$\mathbb{A}_t(\psi(\mathbf{x})) = \{\mathbf{v} \in \mathbb{X}^t \mid \psi(\mathbf{v}) \in \mathbb{M}_t(\psi(\mathbf{x}))\} \quad (2)$$

where for each $\mathbf{m} = (m_0, \dots, m_{q-1}) \in \mathbb{N}^q$

$$\mathbb{M}_t(\mathbf{m}) = \{(\tilde{m}_0, \dots, \tilde{m}_{q-1}) \in \mathbb{N}^q \mid \sum_{i=0}^{q-1} \tilde{m}_i = t, 0 \leq \tilde{m}_i \leq m_i, m_i \in \mathbf{m}\}$$

doesn't depend on the specific values assumed by the entries of $\mathbf{x}$, but on the multiplicity of those values within it.

It should be noted that the size of this set doesn't depend on the specific values assumed by the entries of $\mathbf{x}$, but on the multiplicity of those values within it. However, given $\mathbf{x}$, estimating the size of the set $\mathbb{A}_t(\psi(\mathbf{x}))$ is not trivial (except when $t = n$), then, we can state the following lemma:

Lemma 1. *Given $\mathbf{x} \in \mathbb{X}$ and a non zero integer $t \leq n$ the set $\mathbb{A}_t(\psi(\mathbf{x}))$ has a cardinality equal to:*

$$|\mathbb{A}_t(\psi(\mathbf{x}))| = \sum_{(\tilde{m}_0, \dots, \tilde{m}_{q-1}) \in \mathbb{M}_t(\psi(\mathbf{x}))} \binom{t}{\tilde{m}_0, \dots, \tilde{m}_{q-1}}$$

Lower bound We describe a lower bound for the number of arrangements but, first, we need some extra notation. For a vector $\mathbf{e} = (e_1, \dots, e_q) \in \mathbb{N}^q$, we define

$$\text{sum}(\mathbf{e}) = \sum_{i=1}^q e_i, \quad \text{wt}(\mathbf{e}) = |\{e_i \neq 0 \mid i = \{1, \dots, q\}\}|.$$

Moreover, for a vector $\mathbf{m} \in \mathbb{N}^q$ and an integer $t \geq 1$, we define the following set:

$$\mathbb{E}_t(\mathbf{m}) = \{\mathbf{m} - \mathbf{e} \in \mathbb{N}^q \mid \mathbf{e} \in \mathbb{N}^q, \text{sum}(\mathbf{e}) = t\}.$$

Notice that the number of elements in $\mathbb{E}_t(\mathbf{m})$ depends on both t and $\mathbf{m}$. We have the following:

Theorem 1. *Let $\mathbf{m} = (m_0, \dots, m_{q-1})$ with $\text{wt}(\mathbf{m}) = d$ and $\text{sum}(\mathbf{m}) = n$. For any integers t, u such that $0 \leq u \leq t \leq n$, we have*

$$\mathbb{A}_u(\mathbf{m}) \cdot \mathbb{A}_{t-u}(\mathbf{m}^*) \geq \mathbb{A}_t(\mathbf{m}) \geq \mathbb{A}_u(\mathbf{m}) \cdot \mathbb{A}_{t-u}(\overline{\mathbf{m}}),$$

$\overline{\mathbf{m}}, \mathbf{m}^* \in \mathbb{E}_t(\mathbf{m})$ are such that

$$\forall \mathbf{m}' \in \mathbb{E}_t(\mathbf{m}), \mathbf{m}' \neq \mathbf{m}^*: \quad \mathbb{A}_{t-u}(\mathbf{m}') \leq \mathbb{A}_{t-u}(\mathbf{m}^*),$$

$$\forall \mathbf{m}' \in \mathbb{E}_t(\mathbf{m}), \mathbf{m}' \neq \overline{\mathbf{m}}: \quad \mathbb{A}_{t-u}(\mathbf{m}') \geq \mathbb{A}_{t-u}(\overline{\mathbf{m}}).$$

3 SPECK Interactive Protocol

In this Section we introduce the interactive version of SPECK, that we call SPECK-ID, and prove its security.

3.1 Permutation Equivalence of Codes and Kernel

As we have already recalled, verifying a solution for PEP requires to compute systematic forms, which takes time $O(n^3)$. In contrast, verifying a solution for PKP is significantly faster, requiring only a single vector-matrix multiplication to check if a permuted vector lies within the kernel of a given matrix. However, PEP enjoys a very easy sigma protocol for proving knowledge of a witness [5], a feature notably absent in the current literature for PKP whose only protocols rely on the more complex MPCitH framework [1] or have five rounds [10]. Our main idea is thus to merge these two problems, by using an instance of PEP as a trapdoor for proving the knowledge of the solution of a PKP instance. To do so, we introduce the following problem, whose hardness is discussed in Section 6.

Problem 3 (Permutation Equivalence of Codes and Kernel (PECK))

Given two permutation equivalent codes $\mathcal{C}, \mathcal{C}' \subseteq \mathbb{F}_q^n$ of dimension k , having respectively generator matrix $\mathbf{G}$ and parity-check matrix $\mathbf{H}'$ and a random vector $\mathbf{u} \in \mathbb{F}_q^k$, find a permutation $\mathbf{P} \in S_n$ such that $\mathbf{uGP}\mathbf{H}'^\top = \mathbf{0}$.

In other words, PECK asks to find a permutation $\mathbf{P}$ mapping a random codeword of $\mathcal{C}$ (computed as $\mathbf{uG}$) into $\mathcal{C}'$. Observe that checking a witness for PECK requires just vector-matrix multiplications, which take time $O(n^2)$. In practice, one can even consider that both $\mathbf{G}$ and $\mathbf{H}'$ are in systematic form: this gives a significant speedup in practice.

Remark 1. Throughout this work, we use two interchangeable definitions for PECK instance, namely $(\mathbf{G}, \mathbf{H}', \mathbf{u})$ and $(\mathbf{G}, \mathbf{H}', \mathbf{c})$, where we implicitly define $\mathbf{c} = \mathbf{uG}$. It is clear that these two formulations are equivalent, and thus we will choose the most fitting one according to the context each time.

3.2 Interactive Protocol

We start from the previous considerations to build an interactive protocol based on PECK. We follow the idea sketched in Figure 1. The public key is given by a generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ for $\mathcal{C}$ and a parity-check matrix $\mathbf{H}' \in \mathbb{F}_q^{(n-k) \times n}$ for $\mathcal{C}'$, so that $\mathbf{GPH}'^\top = \mathbf{0}$ for a permutation $\mathbf{P} \in S_n$ which is the secret key. The prover commits to a random codeword of a code $\mathcal{C}^*$ which is equivalent to $\mathcal{C}$. This is done by first sampling a random permutation $\mathbf{P}^* \xleftarrow{\$} S_n$ and a random vector $\mathbf{u} \xleftarrow{\$} \mathbb{F}_q^k$ and then computing $\mathbf{c} = \mathbf{uGP}^*$. When the challenge is 0, the prover responds with $(\mathbf{u}, \mathbf{P}^*)$, so that the verifier can check that indeed the commitment was honestly generated. When the challenge is 1, the prover

responds instead with $\mathbf{P}' = \mathbf{P}^{*-1}\mathbf{P}$, so that the verifier can check that indeed $\mathbf{c}\mathbf{P}'\mathbf{H}'^\top = \mathbf{0}$.

The protocol is given in Figure 3. In the description, we incorporate intuitive optimizations, such as committing to the hash of $\mathbf{c}$ and employing a seed to sample both $\mathbf{u}$ and $\mathbf{P}^*$. We indicate by $\mathcal{O}$ a random oracle that, on input a λ -bit string, returns a uniformly random couple $(\mathbf{u}, \mathbf{P}^*)$ where $\mathbf{u} \in \mathbb{F}_q^k$ and $\mathbf{P}^*$ is a permutation matrix.

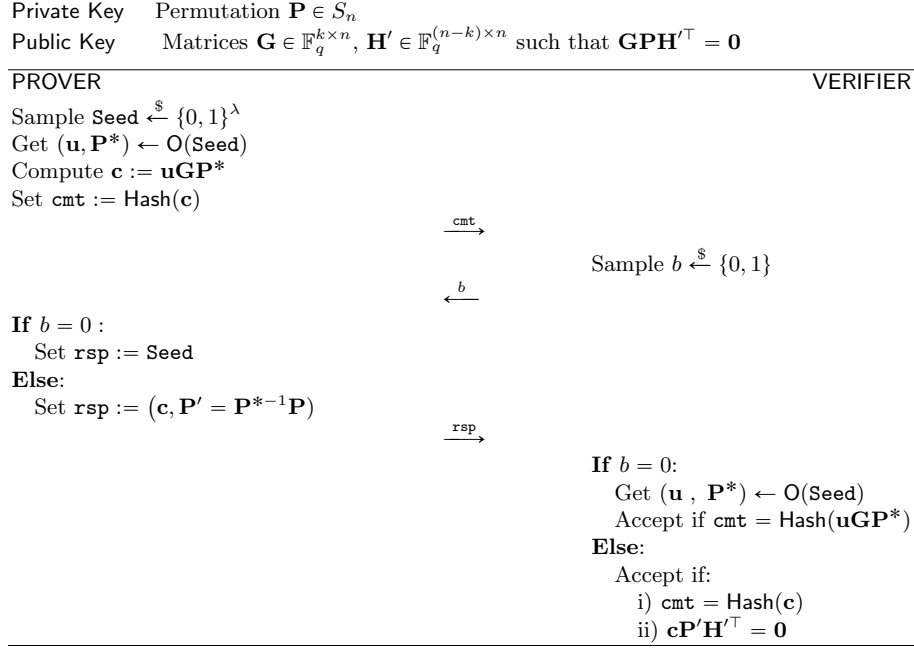


Fig. 3: One round of the new protocol

3.3 Security

We now need to prove the security of our protocol. While it would be tempting to follow the classical paradigm of proving the completeness, honest-verifier zero-knowledge and soundness, it would be impossible to do so: it is easy to show that from two accepting transcripts with same commitment and different challenge one can solve, in polynomial time, a random PKP instance but the private key could be a different permutation. Indeed, let the two transcripts be

$$T = (h, \mathbf{c}, \mathbf{Seed}), \quad \tilde{T} = (h, (\tilde{\mathbf{c}}, \tilde{\mathbf{P}}')).$$

From $\mathbf{Seed}$, one generates $\mathbf{u}$ and $\mathbf{P}^*$ and sets $\mathbf{c} = \mathbf{u}\mathbf{H}\mathbf{P}$. Since h is the same in both transcripts, this guarantees that either hash collisions are found, or indeed

Experiment 1: $\text{Exp}_{\Pi, \mathcal{I}}^{\text{IMP}}(\lambda)$

1: $(x, y) \xleftarrow{\$} R$ 2: $\text{cmt} \xleftarrow{\$} \mathcal{I}^{\text{OTrGen}, \text{O}}(x)$ 3: $\text{ch} \xleftarrow{\$} \text{Ch}$ 4: $\text{rsp} \xleftarrow{\$} \mathcal{I}(\text{cmt}, \text{ch})$ 5: return $\mathcal{V}(\text{pk}, \text{cmt}, \text{rsp})$	$\text{OTrGen}(x)$: return $(\mathcal{P}(y), \mathcal{V})(x)$
--	--

$\mathbf{c} = \tilde{\mathbf{c}}$. Knowing $\mathbf{P}^*$ and $\tilde{\mathbf{P}}'$, one can compute

$$\bar{\mathbf{P}} = \mathbf{P}^* \tilde{\mathbf{P}}',$$

which is granted to send some codewords of $\mathbf{G}$ to the code whose parity-check matrix is $\mathbf{H}'$. However, this is only showing that a successful cheating prover is able to solve random PKP instances. This is a presumed hard problem, but this does not guarantee that the whole code $\mathbf{G}$ is sent to $\mathbf{G}'$ via $\mathbf{P}$. It is worth noting, that it is possible to choose q such that the expected number of permutations sending $\mathbf{c}$ to the kernel of $\mathbf{H}'$ is 1 with overwhelming probability (what we called High q regime in the Introduction), then this implies that $\bar{\mathbf{P}}$ must be equal to the secret $\mathbf{P}$. In such a case, the protocol is a proof of knowledge and, in particular, it is special sound (the proof follows trivially from the previous considerations). We show exactly how to choose q and how its value is linked to the expected number of solutions in Section 6.

However, even when this is not the case, it is still possible to prove that the signature obtained by applying the Fiat-Shamir Transform is secure. Indeed, the sufficient and necessary conditions for having an EUF-CMA signature is that the starting sigma protocol is complete and secure against impersonation under passive attacks [2]. In particular, the following theorem holds:

Theorem 2. [2, Theorem 3.3] *Let Π be a three-move public coin interactive proof with super-polynomially large commitment space, and let $\text{FS}(\Pi)$ be the associated signature obtained via the Fiat-Shamir Transform. Then $\text{FS}(\Pi)$ is polynomially secure against chosen-message attacks in the random oracle model if and only if Π is polynomially-secure against impersonation under passive attacks.*

Definition 1 (Security against impersonation under passive attack) *Let Π be a canonical identification scheme and let $\mathcal{I}$ be an impersonator and λ be the security parameter. We define the advantage of $\mathcal{I}$ playing the IMP experiment $\text{Exp}_{\Pi, \mathcal{I}}^{\text{IMP}}(\lambda)$ (Experiment 1) against Π as:*

$$\mathcal{A}_{\Pi, \mathcal{I}}^{\text{IMP}}(\lambda) = \Pr(\text{Exp}_{\Pi, \mathcal{I}}^{\text{IMP}}(\lambda) = 1)$$

We say that Π is polynomially secure against impersonations under passive attack if $\mathcal{A}_{\Pi, \mathcal{I}}^{\text{IMP}}(\lambda)$ is negligible for every probabilistic polynomial time impersonator $\mathcal{I}$.

The completeness of the protocol is straightforward, we now prove its security against impersonation attack. As a side result of the proof, we also obtain that the protocol is honest-verifier zero-knowledge.

Proposition 1. *If the PECK problem is hard then the protocol of Figure 3 is secure against impersonation under passive attacks.*

Proof. We show that if an adversary $\mathcal{I}$ is able to win the impersonation game, then there exist an adversary $\mathcal{A}$ that solves PECK. Let $\mathbf{G}, \mathbf{H}', \tilde{\mathbf{u}}$ be an instance of PECK, $\mathcal{A}$ sets $(\mathbf{G}, \mathbf{H}')$ as the public key of SPECK-ID and starts the $\mathcal{I}$ of the experiment $\text{Exp}_{\text{SPECK-ID}, \mathcal{I}}^{\text{IMP}}(\lambda)$.

At this point $\mathcal{A}$ needs to provide to $\mathcal{I}$ the transcripts of line 3. For every transcript, $\mathcal{A}$ picks a random challenge $\mathbf{ch} \in \{0, 1\}^\lambda$. Then depending on each bit $\mathbf{ch}_i$ it does:

- If $\mathbf{ch}_i = 0$, $\mathcal{A}$ picks a random seed $\mathbf{Seed}$, picks random $\mathbf{u}, \mathbf{P}^*$ and set $\mathbf{O}(\mathbf{Seed}) = (\mathbf{u}, \mathbf{P}^*)$, then it follows the protocol normally. $\mathcal{A}$ saves the couple $(\mathbf{Seed}, (\mathbf{u}, \mathbf{P}^*))$ in a table $\mathbf{HT}$.
- If $\mathbf{ch}_i = 1$, the adversary picks a random $\mathbf{c}'$ such that $\mathbf{c}' \mathbf{H}'^\top = \mathbf{0}$. Then, it picks a random $\mathbf{P}^*$ and computes $\mathbf{c} = \mathbf{c}' \mathbf{P}^{*-1}$.

It is easy to see that in both cases the transcript verifies and has the same distribution of an honest generated transcript. Indeed:

- if $b = 0$ $\mathcal{A}$ acts exactly as an honest prover and thus the simulation is perfect.
- if $b = 1$ the only difference is that the adversary does not know $\mathbf{Seed}$ such that $(\mathbf{u}, \mathbf{P}^*) \leftarrow \mathbf{O}(\mathbf{Seed})$ and $\mathbf{c} = \mathbf{u} \mathbf{P}^* \mathbf{G}$. However, since $\mathbf{Seed}$ is λ -bit long the verifier can guess it only with negligible probability $\frac{1}{2^\lambda}$.

At this point the impersonator starts the impersonation attempt, producing $\mathbf{cmt}$. We argue that, in order to have more than negligible probability of winning the impersonation game, $\mathcal{I}$ must queries the random oracle $\mathbf{O}$ for at least one parallel execution: indeed, let us suppose that the $\mathcal{I}$ never queries the random oracle to obtain $(\mathbf{u}, \mathbf{P}^*)$, then the impersonation attempt would fail with overwhelming probability, namely $1 - \frac{1}{2^\lambda}$, which is the probability of getting as challenge the string 1^λ . Moreover we argue that $\mathcal{I}$ makes at least an oracle query for a parallel repetition j that will have challenge $\mathbf{ch}_j = 1$: indeed, suppose that $\mathcal{I}$ successfully performs the impersonation attacks only when all its random oracle query has challenge 0, then its success probability is bounded by

$$\Pr(J \leftarrow \mathcal{I}^{\mathbf{OTrGen}, \mathbf{O}}, \mathbf{ch} \xleftarrow{\$} \{0, 1\}^\lambda, \mathbf{ch}_{j \notin J} = 1, \mathbf{ch}_{j \in J} = 0) = \frac{1}{2^\lambda}$$

Since $\mathbf{ch}$ is sampled uniformly at random after the generation of $\mathbf{cmt}$. Since $\frac{1}{2^\lambda}$ is negligible this would imply that $\mathcal{I}$ has negligible success probability, which is a contradiction.

Thus we can suppose that $\mathcal{I}$ perform at least one random oracle queries in the impersonation that corresponds to challenge 1.

We now show how $\mathcal{A}$ answers the random oracle queries made by $\mathcal{I}$:

- if the impersonator queries $\mathbf{O}$ on input $\mathbf{Seed}$ such that there exist an entry $(\mathbf{Seed}, \cdot) \in \mathbf{HT}$, then $\mathcal{A}$ answer in the same way as in the hash table.
- otherwise $\mathcal{A}$ picks a random $\mathbf{u}, \mathbf{P}^*$ and sends them.

In this way $\mathcal{A}$ answers all the queries except one, where it picks a random permutation $\tilde{\mathbf{P}}$ and answer with $\tilde{\mathbf{u}}, \tilde{\mathbf{P}}$. We refer to this query as the *special query*. Notice that this query is indistinguishable from all the other ones, since the only difference is that $\mathcal{A}$ does not pick $\tilde{\mathbf{u}}$ by itself but instead uses the one received at the beginning, however by definition $\tilde{\mathbf{u}}$ is uniformly distributed in $\mathbb{F}_q^k$.

Then, when receiving the commitment $\mathbf{cmt}$, $\mathcal{A}$ picks a random challenge $\mathbf{ch}$ and sends it to $\mathcal{I}$, getting back the response $\mathbf{rsp}$.

Now we bound the probability that $\mathcal{A}$ is able to solve the received PECK instance received at the beginning using $\mathbf{rsp}$. In particular, in order to win all these event must happens:

- The impersonator $\mathcal{I}$ succeeds in its impersonator attempt, which happens with non-negligible probability ϵ .
- $\mathcal{I}$ uses the special query. If Q is the (polynomial) number of random-oracle queries made by $\mathcal{I}$ this happens with probability at least $\frac{1}{Q}$.
- the challenge bit $\mathbf{ch}_i$ that correspond to the repetition in which the special query is used is 1. This happens with probability $\frac{1}{2}$, since the challenge is randomly chosen.

Thus, the success probability of $\mathcal{A}$ is $\frac{\epsilon}{2Q}$ which is non negligible. $\square$

Remark 2. We avoided to explicitly prove that SPECK-ID is honest-verifier zero-knowledge, since it is not necessary for proving the EUF-CMA security of the signature, as per Theorem 2, however it is easy to notice that by using the same strategy used by $\mathcal{A}$ to produce the transcript, it is possible to prove that SPECK-ID is also honest-verifier zero-knowledge.

As noted before, in the High q regime the protocol is also special sound, thus in this setting SPECK-ID is a sigma protocol.

4 SPECK Signature Scheme

In this section we provide a detailed description of the SPECK signature scheme. We follow the traditional approach of applying the Fiat-Shamir transform to the interactive SPECK-ID protocol we presented in Section 3. But, before we do this, we introduce specific optimizations to reduce the communication cost of the protocol. Then, we present full details for the resulting signature scheme and provide numerical results about the implementation.

4.1 Reducing the communication cost

Observe that the SPECK-ID protocol shown in Figure 3 has responses of size λ when the challenge bit is $b = 0$ (i.e. the length of the seed $\mathbf{Seed}$), while for

$b = 1$ the response gets much larger, Indeed, the prover sends both $\mathbf{c} \in \mathbb{F}_q^n$ and $\mathbf{P}' \in S_n$, for an overall binary size of $n \lceil \log_2(q) \rceil + n \lceil \log_2(n) \rceil$. Notice that, here, we are encoding the permutation as a length- n list of integers, each in the range $\{1, \dots, n\}$. While more efficient encoding techniques for a permutation may be employed, they would not change drastically the binary size (and, actually, would introduce some additional computational overhead). We instead rewrite the protocol, aiming for the possibility of providing a more compact response for the case $b = 1$.

Committing to a canonical representative Following an idea similar to [14], we rely on canonical representatives for certain equivalence classes induced by S_n acting on $\mathbb{F}_q^n$. To do this, we define an order $<_q$ on $\mathbb{F}_q$ and, then we use it to define the lexicographical order $\stackrel{\text{Lex}}{<}$ over $\mathbb{F}_q^n$: we say that for two vectors $\mathbf{a} = (a_1, \dots, a_n)$ and $\mathbf{b} = (b_1, \dots, b_n)$ it holds that $\mathbf{a} \stackrel{\text{Lex}}{<} \mathbf{b}$ if there exists an index $i \in \{1, \dots, n\}$ so that $a_j = b_j$ for every $j < i$ and $a_i <_q b_i$.

Since we are considering only the case of q a prime, we rely on the “standard” order given by embedding $\mathbb{F}$ into $\mathbb{N}$. In other words, we represent the finite field $\mathbb{F}_q$ as $\mathbb{F}_q = \{0, 1, \dots, q-1\}$ and consider that $0 <_q 1 <_q \dots <_q q-1$.

We are now ready to formally define what we mean by canonical representatives and equivalence classes.

Definition 2 (Canonical Representatives for vectors) *For a vector $\mathbf{v} \in \mathbb{F}_q^n$, we define its equivalence class, under the action of S_n , as the set*

$$\mathfrak{C}(\mathbf{v}) = \{ \mathbf{v}' \in \mathbb{F}_q^n \mid \exists \mathbf{P} \in S_n \text{ such that } \mathbf{v}' = \mathbf{v}\mathbf{P} \}.$$

Then, we say that a vector $\mathbf{x}$ is a canonical representative for $\mathfrak{C}(\mathbf{v})$ if

$$\mathbf{x} \stackrel{\text{Lex}}{<} \mathbf{v}', \quad \forall \mathbf{v}' \in \mathfrak{C}(\mathbf{v}).$$

We denote by $\text{LexMin}(\mathbf{v})$ the function that, on input a vector $\mathbf{v} \in \mathbb{F}_q^n$, returns the canonical representative for its equivalence class. Relying on the lexicographical ordering we have defined, this function works simply by lifting $\mathbf{v}$ to $\mathbb{N}^n$ and then sorting its elements, which requires $O(n \log n)$ operations. It is easy to see that this function is always well defined and returns a canonical representative according to Definition 2.

We now describe how canonical representatives can be exploited to reduce the signature size. First of all, notice that when the challenge bit is $b = 1$, it is possible for the prover to send $(\mathbf{c}, \mathbf{y} = \mathbf{c}\mathbf{P}')$ instead of $(\mathbf{c}, \mathbf{P}')$. Indeed, the verifier can easily check that i) $\mathbf{y}$ is in fact a permutation of $\mathbf{c}$, and ii) $\mathbf{y}\mathbf{H}'^\top = \mathbf{0}$.

While not particularly helpful on its own, this idea is the starting point of our first optimization. We indeed show that, with the following modifications to the protocol, the vector $\mathbf{c}$ may actually not be provided at all:

- the prover first computes $\mathbf{c} = \mathbf{u}\mathbf{G}\mathbf{P}^*$ as in the base protocol and then sets $\mathbf{x} = \text{LexMin}(\mathbf{c})$. The commitment is $\text{cmt} = \text{Hash}(\mathbf{x})$;

- if $b = 0$, the response is the same as the base protocol: the verifier samples $(\mathbf{u}, \mathbf{P}^*)$ and checks that $\text{cmt} = \text{Hash}(\text{LexMin}(\mathbf{uGP}^*))$;
- when $b = 1$, the prover responds with $\mathbf{y} = \mathbf{cP}' = \mathbf{cP}^{*-1}\mathbf{P}$. The verifier then checks that $\mathbf{yH}'^\top = \mathbf{0}$ and $\text{cmt} = \text{Hash}(\text{LexMin}(\mathbf{y}))$. Indeed, by the property of LexMin , this ensures that either the prover found a collision for Hash , or that the value the prover committed to is indeed a permutation of $\mathbf{y}$.

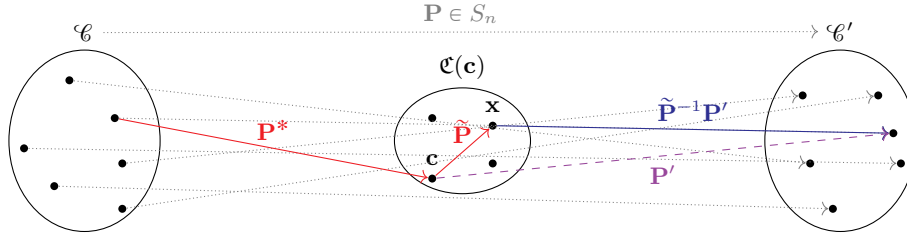


Fig. 4: Representation of the ZK protocol using canonical representatives. The arrows in red correspond to what the verifier checks for the case $b = 0$, while the blue arrow corresponds to the permutation that the verifier checks when $b = 1$; $\tilde{\mathbf{P}}$ is a permutation sending $\mathbf{c}$ to $\mathbf{x}$, the canonical representative for the equivalence class.

Avoiding the ephemeral permutation In the original protocol, the permutation $\mathbf{P}^*$ was crucial to hide the secret permutation $\mathbf{P}$ when the challenge is $b = 1$. However, with the usage of canonical representatives, this is no longer needed. Indeed, notice that both $\text{LexMin}(\mathbf{uGP}^*) = \text{LexMin}(\mathbf{uG})$ regardless of $\mathbf{P}^*$, making the multiplication by $\mathbf{P}^*$ pointless since both computation leads to the same hash. Thus, the verifier can just check that $\text{cmt} = \text{LexMin}(\mathbf{uG})$, since $\mathbf{uG}$ and $\mathbf{uGP}^*$ belong to the same equivalence class. For the same reason, when preparing the commitment, the prover can just compute and commit to $\text{LexMin}(\mathbf{uG})$. When replying to challenge $b = 1$, the prover just provide $\mathbf{c}' = \mathbf{uGP}$. While this modification does not improve the communication cost, since $\mathbf{P}^*$ is always derived from a seed **Seed**, this reduces the number of operations needed during both the signing and verification algorithms.

Notice that if $\text{LexMin}(\mathbf{uG}) = \mathbf{uG}$ the response is exactly the secret key. However this happens with negligible probability. Indeed, a necessary condition for $\mathbf{uG}$ to be in lexicographical order is that $\mathbf{u}$ is in lexicographical order, since $\mathbf{G}$ is in systematic form. Thus we have

$$\Pr(\text{LexMin}(\mathbf{uG}) = \mathbf{uG}) \leq \frac{\binom{k+q-1}{k}}{q^k}$$

where $\binom{k+q-1}{k}$ is the number of vector in $\mathbb{F}_q^k$ in lexicographical order, counted as the number of k -combinations of q elements with repetitions. Using $q = 127$ and $k = 126$ we have that

$$\frac{\binom{k+q-1}{k}}{q^k} \leq 2^{-363}.$$

Removing redundancy For a code of dimension k , in every codeword, only k coordinates (a so called information set) bring information, while the other $n - k$ symbols bring “redundancy” and can be uniquely computed once the remaining k are known. We can leverage this fact to further reduce the communication cost when $b = 1$. Let $\mathbf{H}' = (\mathbf{B}, \mathbf{I}_{n-k})$ be a parity-check matrix in systematic form. If $\mathbf{y} = (\mathbf{y}_1, \mathbf{y}_2)$, with $\mathbf{y}_1 \in \mathbb{F}_q^k$ and $\mathbf{y}_2 \in \mathbb{F}_q^{n-k}$, is a codeword of the code having $\mathbf{H}'$ as parity-check matrix, then

$$\mathbf{y}\mathbf{H}'^\top = \mathbf{0} \implies \mathbf{y}_1\mathbf{B}^\top + \mathbf{y}_2 = \mathbf{0} \implies \mathbf{y}_2 = -\mathbf{y}_1\mathbf{B}^\top.$$

That is, we interpret $\mathbf{y}_2$ and the redundancy symbols of $\mathbf{y}$ and recover them from the sole knowledge of the information symbols $\mathbf{y}_1$. Thus, for the case $b = 1$, the prover can just send $\mathbf{y}_1$:

- the prover first computes $\mathbf{y} = \mathbf{c}\mathbf{P}$, then responds only with $\mathbf{y}_1$, that is, with the first k symbols of $\mathbf{y}$;
- the verifier computes $\mathbf{y}_2 = -\mathbf{y}_1\mathbf{M}^\top$, sets $\mathbf{y} = (\mathbf{y}_1, \mathbf{y}_2)$ and then checks that $\text{cmt} = \text{Hash}(\text{LexMin}(\mathbf{y}))$.

Notice that in this case the verifier does not need to verify that the received codeword is in the kernel of $\mathbf{H}'$ since $(\mathbf{y}_1, \mathbf{y}_2)$ is guaranteed to be in it by construction, thus it does not increase the computational complexity of the verification process. Moreover, this modification further reduces the communication cost for the case $b = 1$ down to $k \log_2(q)$.

Remark 3. We now discuss the security of the above optimisations. It is clear that the last optimisation is secure, since it does not modify the underlying protocol but simply exploit the code for a more efficient encoding. Moreover it is also immediate to prove that as long as using canonical representative is secure also removing the ephemeral permutation is secure, since both executions would result to the same commitment and signature. So it is enough to prove that using canonical representative is secure. This is rather intuitive once one realises that committing to $\text{LexMin}(\mathbf{c})$ instead of $\mathbf{c}$ means applying an additional permutation on $\mathbf{c}$. This is visualised in Figure 4. Replying correctly to both challenges means finding a permutation $\mathbf{P}'$ sending $\mathbf{c}$ into $\mathcal{C}'$, that is, solving a PECK instance.

More formally, we claim that if the hardness of the PECK problem holds, then SPECK-ID with canonical representatives is secure against impersonation under passive attack. With the same notation and strategy of Proposition 1, if a polynomial time impersonator $\mathcal{I}$ exists, given an instance of PECK $(\mathbf{c}, \mathbf{G}, \mathbf{H}')$, we

can build a polynomial time algorithm $\mathcal{A}$ that finds $\mathbf{P}$ such that $\mathbf{cPH}'^\top = \mathbf{0}$. To simulate the hash queries $\mathcal{A}$ use the same strategy as before, while to simulate the sign queries $\mathcal{A}$ does the following:

- If $\text{ch}_i = 0$, $\mathcal{A}$ picks a random seed Seed , picks random $\mathbf{u}, \mathbf{P}^*$ and set $\text{O}(\text{Seed}) = (\mathbf{u}, \mathbf{P}^*)$, then it follows the protocol normally by setting the commitment as the hash of the lexicographical order of $\mathbf{uGP}^*$. $\mathcal{A}$ saves the couple $(\text{Seed}, (\mathbf{u}, \mathbf{P}^*))$ in a table HT.
- If $\text{ch}_i = 1$, the adversary picks a random $\mathbf{c}'$ such that $\mathbf{c}'\mathbf{H}'^\top = \mathbf{0}$. Then, it picks a random $\mathbf{P}^*$ and computes $\mathbf{c} = \mathbf{c}'\mathbf{P}^{*-1}$. Finally it computes $\text{LexMin}(\mathbf{c})$.

It is easy to see that this simulation is indistinguishable from a real execution. Once receiving the forgery, with the same probability of the original game, $\mathcal{A}$ can retrieve a solution for the PECK in the following way. $\mathcal{A}$ receive a permutation mapping $\text{LexMin}(\mathbf{uGP}^*)$ to the second code $\mathcal{C}'$. Moreover $\mathcal{A}$ knows $\mathbf{uG}$ and also $\mathbf{P}^*$ by construction, and thus can retrieve the permutation mapping $\mathbf{uGP}^*$ to its lexicographical order. By multiplying these three permutations $\mathcal{A}$ retrieve the secret key permutations.

Other optimizations When considering t parallel repetitions of the ZK protocol, one can apply further optimizations such as:

- *Fixed weight challenges*: a challenge $b \neq 0$ is used only in w out of t rounds. Using a *puncturable PRF* (such as a GGM tree), the cost of communicating the seeds for all rounds with challenge 0 is not greater than $w\lambda \log_2(t/w) + \text{wt}(t) - 1$, where $\text{wt}(t)$ indicates the Hamming weight of the binary string representing t [12, Proposition 2].
- *A unique commitment*: Let $\text{cmt}^{(i)}$, for $i = 1, \dots, t$, indicate the commitment for round i . Instead of sending each of these values, the prover can more conveniently provide only $\text{cmt} = \text{Hash}(\text{cmt}^{(1)} || \dots || \text{cmt}^{(t)})$.

4.2 The final signature scheme

The SPECK signature scheme is shown in Figure 5. In the description, we have used three additional functions `CreateTree`, `GetSeeds`, `RecoverSeeds` which are used, respectively, to instantiate the GGM tree, get the intermediate seeds and recover the seeds in the bottom layer, given a path of intermediate seeds. To avoid attacks based on commitments collisions, we follow the recommendations from [13] and, in every round, compute the commitment using both the salt and the round index.

Put everything together, the resulting signature size is

$$|\text{Rsp}| \leq \underbrace{4\lambda}_{\text{Salt and commitment}} + \underbrace{\lambda w \log_2(t/w) + \text{wt}(t) - 1}_{\text{Intermediate seeds}} + \underbrace{wk \log_2(q)}_{\text{Responses for rounds with } b^{(i)} = 1}.$$

Public data	Parameters $n, k, q, t, w, s, \lambda \in \mathbb{N}$, hash function $\text{Hash} : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$, hash function $\text{Hash}_{t,w} : \{0, 1\}^* \rightarrow \{\mathbf{b} \in \{0, 1\}^t \mid \text{wt}(\mathbf{b}) = w\}$, functions CreateTree , GetSeeds , RecoverSeeds
Private Key	Permutation $\mathbf{P} \in S_n$
Public Key	Matrices $\mathbf{G} \in \mathbb{F}_q^{k \times n}$, $\mathbf{M} \in \mathbb{F}_q^{(n-k) \times k}$ such that $\mathbf{G}\mathbf{P}(\mathbf{M}, \mathbf{I}_{n-k})^\top = \mathbf{0}$.
PROVER	VERIFIER
Sample $\text{salt} \xleftarrow{\$} \{0, 1\}^{2\lambda}$, $\text{MSeed} \xleftarrow{\$} \{0, 1\}^\lambda$ Set $\{\text{seed}^{(i)}\}_{1 \leq i \leq t} := \text{CreateTree}(\text{MSeed}, \text{Salt}, \text{Msg})$ // Prepare commitments for all rounds For $i = 1, \dots, t$: Sample $\mathbf{u}^{(i)} \xleftarrow{\text{seed}^{(i)}} \mathbb{F}_q^k$ Compute $\mathbf{c}^{(i)} := \mathbf{u}^{(i)}\mathbf{G}$, $\mathbf{x}^{(i)} := \text{Lex}(\mathbf{c}^{(i)})$ Set $\text{cmt}^{(i)} := \text{Hash}(\mathbf{x}^{(i)} \parallel \text{Salt} \parallel i \parallel \text{Msg})$ // Sample challenge vector Set $\text{cmt} := \text{Hash}(\text{cmt}^{(1)} \parallel \dots \parallel \text{cmt}^{(t)})$ Set $(b^{(1)}, \dots, b^{(t)}) := \text{Hash}_{t,w}(\text{cmt})$ Set $J := \{i \in \{1, \dots, t\} \mid b^{(i)} \neq 0\}$ // Get path to reconstruct seeds Set $\text{Seeds} := \text{GetSeeds}(\text{MSeed}, \text{Salt}, \text{Msg}, J)$ // Get responses for rounds with challenge $\neq 0$ For $i \in J$: Compute $\mathbf{y}^{(i)} := (\mathbf{y}_1^{(i)}, \mathbf{y}_2^{(i)}) = \mathbf{c}^{(i)}\mathbf{P}^{(i)}$ // Prepare and output signature Set $\text{Rsp} := (\text{Salt}, \text{cmt}, \text{Seeds}, \{\mathbf{y}_1^{(i)}\}_{i \in J})$	
	// Sample challenge vector Set $(b^{(1)}, \dots, b^{(t)}) := \text{Hash}_{t,w}(\text{cmt} \parallel \text{Salt} \parallel \text{Msg})$ // Recover seeds for rounds with challenge = 0 Set $J := \{i \in \{1, \dots, t\} \mid b^{(i)} \neq 0\}$ Compute $\{\text{seed}^{(i)}\}_{i \notin J} := \text{RecoverSeeds}(\text{Seeds}, \text{Salt}, \text{Msg}, J)$ // Recompute $\text{cmt}^{(i)}$ for each round For $i = 1, \dots, t$: If $b^{(i)} = 0$: // Use $\text{seed}^{(i)}$ Sample $\mathbf{u}^{(i)} \xleftarrow{\text{seed}^{(i)}} \mathbb{F}_q^k$, compute $\mathbf{c}^{(i)} := \mathbf{u}^{(i)}\mathbf{G}$ Compute $\mathbf{x}^{(i)} := \text{LexMin}(\mathbf{c}^{(i)})$ Else: Compute $\mathbf{y}_2^{(i)} := -\mathbf{y}_1^{(i)}\mathbf{M}^\top$, set $\mathbf{y}^{(i)} := (\mathbf{y}_1^{(i)}, \mathbf{y}_2^{(i)})$ Compute $\mathbf{x}^{(i)} := \text{LexMin}(\mathbf{y}^{(i)})$ // Get canonical representative for round i Set $\text{cmt}^{(i)} := \text{Hash}(\mathbf{x}^{(i)} \parallel \text{Salt} \parallel i \parallel \text{Msg})$ // Check validity of cmt Accept if $\text{cmt} = \text{Hash}(\text{cmt}^{(1)} \parallel \dots \parallel \text{cmt}^{(t)})$

Fig. 5: The SPECK signature scheme

Remark 4. Since the scheme is *commitment-recoverable*, we may save some bits (say, approximately λ bits) in the signature including the challenge instead of cmt . Verification would work by recomputing the challenge, using $\text{cmt}^{(1)}, \dots, \text{cmt}^{(t)}$ computed during verification. We do not rely on this optimization since it would require an ad-hoc encoding for the challenge vector which, in turn, would require to dispose of an efficient encoding from binary strings to fixed weight vectors. This would increase the computational burden, at the sake of a very modest reduction in the signature size.

Instance	q	Pk size (kB)	(t, w)	Signature size (kB)
Speck-Low – 133 – 60	127	27.8	(133, 60)	7.8
Speck – Low – 256 – 30			(256, 30)	4.9
Speck – Low – 512 – 23			(512, 23)	4.2
Speck – Low – 768 – 20			(768, 20)	3.9
Speck – Low – 4096 – 14			(4096, 14)	3.4
Speck – High – 192 – 36	8861	55.5	(192, 36)	8.9
Speck – High – 133 – 60			(256, 30)	7.7
Speck – High – 512 – 23			(512, 23)	6.5
Speck – High – 768 – 20			(768, 20)	5.9
Speck – High – 4096 – 14			(4096, 14)	4.8

Table 1: Parameters, signature and public key sizes for some recommended SPECK instances. Resampling the public key allows for smaller key but higher signature times.

Choosing parameters The parameters q , n and k depend on the hardness of PEP and PECK: this matter is discussed deeply in Section 6. The values of t and w shall be such that $\binom{t}{w} > 2^\lambda$. Indeed, this grants that the number of challenges is greater than 2^λ . Thanks to the Fiat-Shamir heuristic, forgeries boil down to guessing the challenge value, hence we pick these parameters so that this takes, on average, the challenge space has size at least 2^λ .

4.3 Performances

In Table 1 we recommend some instances for SPECK. The code parameters are the same as LESS Category 1, namely, $n = 252$, $k = 126$. For the finite field size, we use $q = 127$ for the Low q regime, and $q = 8861$ for the Large q regime. For each regime, we consider several value of t and w . Large values of t lead to smaller signatures but, obviously, increase latency since more rounds shall be executed and, moreover, the number of leaves in the GGM tree increases.

To verify the computational advantages that we expect from the proposed approach, we provide benchmark results for the optimized implementation of SPECK. The implementation of the scheme has been devised starting from the implementation of LESS. Timings are given in Table 2.

5 Using multiple codes in the public key

Among the optimizations we considered for SPECK, we have not used the “popular” optimization based on giving multiple elements in the public key. This optimization can generically be used for any sigma protocol based on group actions and maintains special soundness while boosting soundness (i.e., reducing the soundness error); the price to pay is in the increased public key size.

Instance	KeyGen		Sign		Verify	
	ms	MCycles	ms	MCycles	ms	MCycles
Speck – Low – 133 – 60	1.91	4.19	1.06	2.33	0.95	2.09
Speck – Low – 256 – 30	1.91	4.19	1.99	4.38	1.93	4.25
Speck – Low – 512 – 23	1.93	4.26	3.98	8.74	3.93	8.65
Speck – Low – 768 – 20	1.92	4.22	5.94	13.12	5.96	13.11
Speck – Low – 4096 – 14	1.91	4.22	31.62	70.08	31.60	69.68

Table 2: Timings for the SPECK instances in the Low q regime without resamplings of the public key. Timings were measured on an Intel(R) Core(TM) i5-10310U clocked at 1.7 GHz with Turbo Boost disabled and are given both as ms and MCycles, as averages of 128 runs.

Since SPECK is derived from a sigma protocol based on the code equivalence group action, at a first glance one may think that the very same optimization can be applied to SPECK as well. Namely, this would correspond to publish $s > 2$ codes $\mathcal{C}_0, \mathcal{C}_1, \dots, \mathcal{C}_{s-1} \subseteq \mathbb{F}_q^n$ so that $\mathcal{C}_i = \pi_i(\mathcal{C}_0)$ with $\pi_i \in S_n$ being a secret permutation, for each i . In each round, the challenge b is sampled from $\{0, \dots, s-1\}$: when $b = 0$, the prover shows that the commitment is indeed a permutation of a codeword coming from $\mathcal{C}_0$ (as in the standard protocol) while, if $b \neq 0$, the prover provides a permutation mapping the commitment into $\mathcal{C}_b$.

Usually, this would lead to a soundness error of $\frac{1}{s}$, however we show that this is not the case for SPECK, both in the Large and Low q regimes, and provide some attacks that make the security sketchy and difficult to assess.

A crucial fact for PECK security is that the vector $\mathbf{u}$ should be randomly chosen, and the verifier checks so only when $b = 0$, while when $b \neq 0$ it is impossible to check any malicious behaviour in that sense (i.e. the adversary can choose an “ad-hoc” vector that gives him some advantages without being caught). So, if there exists a way to choose the committed vector $\mathbf{u}$ so that it is easy to answer to all the $s-1$ non-zero challenges, an adversary using it would be caught only with probability $\frac{1}{s}$ on each iteration. We devote the following Sections to describe in details this kind of attack.

5.1 A powerful cheating strategy (with a trivial patch)

Let us consider one round of the SPECK-ID protocol with $s > 2$ codes in the public key; see Figure 6 for a sketch of how the protocol works. To simplify the treatment, we omit the optimization based on sending only information symbols (the attack we describe can be easily tweaked to account also for this optimization). In the figure, we denote with $\mathbf{y}_i \in \mathbb{F}_q^n$ the response for challenge $b = i$ and with τ_i the permutation that maps $\mathbf{y}_i$ into $\mathcal{C}_i$.

We want to understand what an adversary should do, in order to reply correctly to two or more non-zero challenges.

Let us assume that, for two indices $i \neq j$, the adversary is able to find at least a pair of codewords, one from $\mathcal{C}_i$ and one from $\mathcal{C}_j$, having the same entries

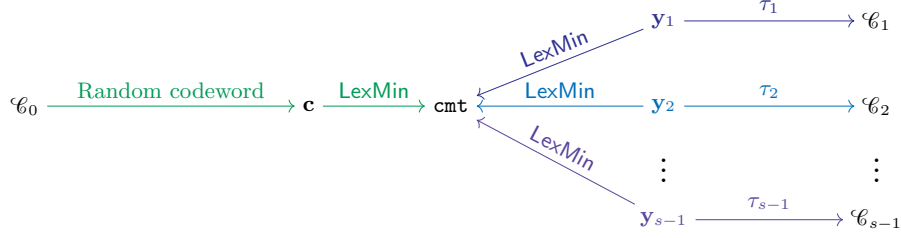


Fig. 6: Representation of the protocol with $s > 2$ codes in the public key.

multiset. It is not hard to see that this can be rewritten as

$$\exists \mathbf{z} \in \mathcal{C}_i, \exists \zeta \in S_n \text{ such that } \zeta(\mathbf{z}) \in \mathcal{C}_j. \quad (3)$$

In such a case, the adversary can cheat by committing to $\text{LexMin}(\mathbf{z})$. If challenged with i , the adversary responds with $\mathbf{y}_i := \mathbf{z}$ while, if challenged with j , the adversary responds with $\mathbf{y}_j := \zeta(\mathbf{z})$. By construction, the adversary will pass verification in both cases.

When $\mathbf{z}$ and ζ as in Equation (3) can be found, one can devise a cheating strategy that succeeds with probability at least $2/s$. Moreover, this probability may be increased by generalising Equation (3) and considering more than two codes.

As the careful reader may have already noticed, Equation (3) actually admits always a solution for any pair of indices i, j : one can just choose $\mathbf{z} = \mathbf{0}$. Indeed, the null codeword is in each possible code, i.e., $\mathbf{0} \in \mathcal{C}_1 \cap \mathcal{C}_2 \cap \dots \cap \mathcal{C}_{s-1}$. Hence, the success probability of this strategy is actually $1 - \frac{1}{s}$: for any challenge $b \neq 0$, replying with $\mathbf{y}_b = \mathbf{0}$ would result in a valid response.

A trivial patch Observe that the attack we have described above can be easily countered if one requires that, for honest signatures, the response in each round is different from the null vector. However, as we show in the next section, there is a subtle reason why this attack exists and this simple patch does not exclude that other types of attacks can be devised. Indeed, it is not hard to see that finding a pair $(\mathbf{z}, \zeta)$ that satisfies Equation (3) implies solving a variant of PECK in which, instead of receiving $\mathbf{uG}$ with $\mathbf{u} \xleftarrow{\$} \mathbb{F}_q^k$, one is free to choose $\mathbf{u}$ as they desire. This allows to select the values of $\mathbf{u}$ for which finding ζ is easy.

We point out that this is a significant difference with PECK, in which we require that $\mathbf{u} \xleftarrow{\$} \mathbb{F}_q^k$. Indeed, this means that even if weak choices for $\mathbf{u}$ exist, the probability that one of such choices is selected is negligible.

5.2 Refining the strategy

At the best of our knowledge, the problem associated with Equation (3) has never been studied in the literature. Hence, we currently do not know what the

best strategy for finding a pair $(\mathbf{z}, \zeta)$ should be. However, we can consider again a trivial solution to Equation (3) by setting ζ equal to the identity. Now, we have two codes $\mathcal{C}_i, \mathcal{C}_j \subseteq \mathbb{F}_q^n$ with the same dimension k , and are searching for a non null codeword $\mathbf{z}$ which belongs to both codes. Such a codeword can be determined efficiently with basic linear algebra, so the question becomes: how likely it is that such a codeword exists? Notice that a codeword of this type exists whenever the two codes intersect in a space of dimension at least 1. As we show next, for two random codes, the probability to have intersection of dimension ≥ 1 is rather large; the proof is given in Appendix B.

Theorem 3. *Let $\mathcal{A}, \mathcal{B} \subseteq \mathbb{F}_q^n$ be two random codes with dimension $k \leq n/2$. Then, the probability that their intersection is trivial (i.e., contains only the null vector) is*

$$\frac{q^{k^2} \cdot \begin{bmatrix} n-k \\ k \end{bmatrix}_q}{\begin{bmatrix} n \\ k \end{bmatrix}_q},$$

where $\begin{bmatrix} m \\ r \end{bmatrix}_q = \prod_{i=0}^{r-1} \frac{q^{m-i} - 1}{q^{r-i} - 1}$.

Remark 5. As a good approximation of the above probability, we can use the probability that a random $2k \times n$ matrix over $\mathbb{F}_q$, with $k \leq n/2$, has full rank: this is given by $q^{-2kn} \prod_{i=0}^{2k} (q^n - q^i)$. We can view this matrix as the concatenation of two random $k \times n$ matrices $\mathbf{G}$ and $\mathbf{G}'$, which stand for the generator matrices of $\mathcal{A}$ and $\mathcal{B}$. If $\begin{pmatrix} \mathbf{G} \\ \mathbf{G}' \end{pmatrix}$ has some rank deficiency, this can be due to i) $\mathbf{G}$ being rank deficient, ii) $\mathbf{G}'$ being rank deficient, or iii) the space generated by $\mathbf{G}$ intersecting that of $\mathbf{G}'$. The first two events are much more unlikely than the third one, since with overwhelming probability the matrices $\mathbf{G}$ and $\mathbf{G}'$ have full rank. In particular, for codes of rate $1/2$, the probability we are looking for is simply the probability that a random $n \times n$ matrix is non singular, which is well approximated by $1 - 1/q$ when q is sufficiently large.

For SPECK instances in the low q regime (i.e., $q = 127$), we get that the above probability is ≈ 0.99206 , hence, there is a 0.008 probability that two random codes intersect in a space with dimension at least 1. Even if we consider the high q regime, the probability is 0.00011

Notice that this is inherent to the fact that we use codes with rate $1/2$: to make the above probability negligible, either we use an exponentially large q (which would increase significantly the signature size), or we use codes having rate $< 1/2$. In any case, this rules out only the existence of trivial solutions to Equation (3): a reliable study of the exact hardness to solve the problem requires a more careful analysis and we leave it for future works.

6 Security Analysis

As seen in Section 3.3, the resistance against forgeries relies entirely on the security of PECK. In this section we discuss the hardness to solve this problem.

We first formally define $\text{PECK}_{q,n,k}$ and $\text{PEP}_{q,n,k}$ in terms of probability distributions.

Definition 3. We define $\mathcal{D}_{q,n,k}^{\text{PEP}}$ as the distribution that:

1. samples uniformly at random a k -dimensional self-orthogonal code $\mathcal{C} \subseteq \mathbb{F}_q^n$ with generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ and parity-check matrix $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$;
2. samples uniformly at random an invertible matrix $\mathbf{S} \xleftarrow{\$} GL_k$ and a permutation matrix $\mathbf{P} \xleftarrow{\$} S_n$ and computes $\mathbf{H}' = \mathbf{SHP}$;
3. outputs $\{\mathbf{G}, \mathbf{H}'\}$.

Definition 4. We define $\mathcal{D}_{q,n,k}^{\text{PECK}}$ as the distribution that:

- samples $\{\mathbf{G}, \mathbf{H}'\} \leftarrow \mathcal{D}_{q,n,k}^{\text{PEP}}$;
- samples $\mathbf{u} \xleftarrow{\$} \mathbb{F}_q^k$ and computes $\mathbf{c} = \mathbf{uG}$;
- returns $\{\mathbf{c}, \mathbf{G}, \mathbf{H}'\}$.

Using the above distributions, we define:

Problem 4 ($\text{PEP}_{q,n,k}$) Given $\{\mathbf{G}, \mathbf{H}'\} \leftarrow \mathcal{D}_{q,n,k}^{\text{PEP}}$, compute a permutation matrix $\mathbf{P} \in S_n$ such that $\mathbf{GPH}'^\top = \mathbf{0}$.

Problem 5 ($\text{PECK}_{q,n,k}$) Given $\{\mathbf{c}, \mathbf{G}, \mathbf{H}'\} \leftarrow \mathcal{D}_{q,n,k}^{\text{PECK}}$, compute a permutation matrix $\mathbf{P} \in S_n$ such that $\mathbf{cPH}'^\top = \mathbf{0}$.

We first derive a (heuristic) estimate, as well as an upper bound, for the average number of solutions to $\text{PECK}_{q,n,k}$.

Proposition 2. Let $\{\mathbf{c}, \mathbf{G}, \mathbf{H}'\} \leftarrow \mathcal{D}_{q,n,k}^{\text{PECK}}$. Then, the expected number of solutions to the problem is

$$N_{\text{sol}} = 1 + \frac{|\mathbb{A}_n(\mathbf{m})| - 1}{q^{n-k}},$$

where $\mathbf{m} = \psi(\mathbf{c})$. This is always less than $1 + \frac{f(n,q)}{q^{n-k}}$, where

$$f(n,q) = \begin{cases} n! & \text{if } n \leq q, \\ \frac{n!}{([n/q]!)^{q \cdot [n/q] - n} \cdot ([n/q]!)^{n - q \cdot [n/q]}} & \text{if } n > q. \end{cases}$$

Proof. Any permuted version of $\mathbf{c}$ is an element of $\mathbb{A}_n(\mathbf{m})$. We know for sure that, among such elements, one is guaranteed to be in $\mathcal{C}'$; for all the other arrangements, we estimate with $q^{-(n-k)}$ the probability that it belongs to $\mathcal{C}'$. Hence, the average number of solutions can be estimated as

$$1 + \frac{|\mathbb{A}_n(\mathbf{m})| - 1}{q^{n-k}} < 1 + \frac{|\mathbb{A}_n(\mathbf{m})|}{q^{n-k}}. \quad (4)$$

Then, we upper bound $\mathbb{A}_n(\mathbf{m})$ with $f(n,q)$ (see [14, Appendix B]). $\square$

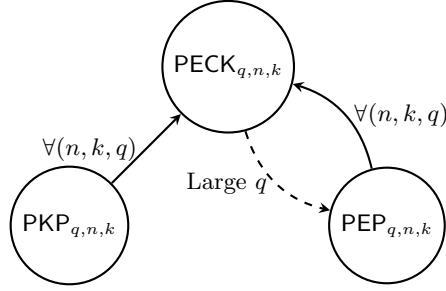


Fig. 7: Reductions between $\text{PECK}_{q,n,k}$, $\text{PEP}_{q,n,k}$ and $\text{PKP}_{q,n,k}$.

6.1 Preliminary considerations

Before studying practical solvers for the problem, we make some high level considerations about the hardness of $\text{PECK}_{q,n,k}$. The relations are summarised in Figure fig. 7 and are motivated in this section.

$\text{PECK}_{q,n,k}$ is easy when q is small We show here that for too small q the problem is easy. For simplicity we limit ourselves to the case $q < n$, if n and q are both small the problem is clearly even easier. Let us consider an algorithm $\mathcal{A}_{\text{PECK}}$ that, on input a given instance $\mathbf{c}, \mathbf{G}, \mathbf{H}'$:

- first samples random codewords from $\mathcal{C}'$ until a codeword $\mathbf{c}'$ with the same entries multiset as $\mathbf{c}$ is obtained,
- then outputs $\mathbf{P} \in S_n$ as a permutation that sends $\mathbf{c}$ to $\mathbf{c}'$.

Since the last step is trivial, the computational cost of this attack corresponds to the average number of codewords $\mathbf{c}'$ that the algorithm needs to sample. Let us simplify by considering that, in $\mathbf{c}$, every field element appears approximately n/q times in $\mathbf{c}$. Our goal is to show that

1. the probability to sample such a codeword $\mathbf{c}$ is high (namely, it decays not faster than a polynomial of n),
2. a polynomial amount of codewords from $\mathcal{C}'$ should be sampled, before a valid $\mathbf{c}'$ is found.

Indeed, if these two conditions hold, by running a polynomial number of instances of $\mathcal{A}_{\text{PECK}}$ in parallel we obtain an efficient solver.

As a warm up, let us consider the case $q = 2$. Deeming the codewords of $\mathcal{C}$ as random vectors over $\mathbb{F}_2^n$, the probability that $\mathbf{c}$ has Hamming weight $n/2$ (half of its entries are 0 and half are 1) is $2^{-n} \cdot \binom{n}{n/2}$. Since $\binom{n}{n/2} \geq 2^n / \sqrt{2n}$, we get that the previous probability is greater than $2^{-n} \cdot \frac{2^n}{\sqrt{2n}} = \frac{1}{\sqrt{2n}}$. With the same probability, a random codeword from $\mathcal{C}'$ has weight $n/2$. Hence, after at most $\sqrt{2n} = O(\sqrt{n})$ attempts, we get a codeword $\mathbf{c}'$ with the desired weight. Thus, we have a solver that requires to sample only $O(\sqrt{n})$ codewords from $\mathcal{C}'$ and that solves a fraction of at least $\approx 1/\sqrt{2n}$ instances.

For the more general case of $q \neq 2$, computations get a bit more involved. In this case, we are able to show (see Appendix C) that, on average, the number of required samples for $\mathbf{c}'$ is

$$N \sim q^n \cdot \frac{\left(\sqrt{2\pi n/q} \cdot \left(\frac{n}{eq}\right)^{n/q}\right)^q}{\sqrt{2\pi n} \left(\frac{n}{e}\right)^n} = (2\pi n)^{\frac{q}{2}-1} \cdot q^{-\frac{q}{2}}.$$

It is clear that when q is small, the total number of queries is low and thus the problem is easy. In the next section we study how to improve the above attack, but first we do some preliminary considerations about the complexity of $\text{PECK}_{q,n,k}$ when compared to $\text{PEP}_{q,n,k}$ and $\text{PKP}_{q,n,k}$.

$\text{PECK}_{q,n,k}$ is at most as hard as $\text{PEP}_{q,n,k}$ This is true for any tuple $\{q, n, k\}$. Indeed, starting from a $\text{PECK}_{q,n,k}$ instance $\{\mathbf{c}, \mathbf{G}, \mathbf{H}'\}$ one can call a solver for $\text{PEP}_{q,n,k}$, on input $\{\mathbf{G}, \mathbf{H}'\}$. The solver will return $\mathbf{P} \in S_n$ which sends $\mathcal{C}$ (the code having $\mathbf{G}$ as generator matrix) to $\mathcal{C}'$ (the code having $\mathbf{H}'$ as parity-check matrix), hence, $\mathbf{P}$ will also send $\mathbf{c}$ to $\mathcal{C}'$.

$\text{PECK}_{q,n,k}$ is as hard as $\text{PEP}_{q,n,k}$ when q is large When the parameters are such that $\text{PECK}_{q,n,k}$ instances admit a unique solution with large probability, then the two problems are expected to be equally hard. First, we determine how large q should be. Recalling Proposition 2, the number of spurious solutions is not greater than $q^{-(n-k)} \cdot (n! - 1) < q^{-(n-k)} \cdot n!$. Asymptotically, we get

$$\frac{n!}{q^{n-k}} \sim \frac{\sqrt{2\pi n} \left(\frac{n}{e}\right)^n}{q^{n(1-R)}} < 1 \implies q > (2\pi n)^{\frac{1}{n(1-R)}} \left(\frac{n}{e}\right)^{\frac{1}{1-R}} > \left(\frac{n}{e}\right)^{\frac{1}{1-R}}. \quad (5)$$

Observe that one can work also with the dual codes, having rate $1 - R$. With the same reasoning, one would get $q > \left(\frac{n}{e}\right)^{\frac{1}{R}}$, which is less than the above bound if $R < \frac{1}{2}$. For this reason, we give the reduction for $R \geq 1/2$ and consider that for $R < 1/2$ it is enough to focus on the dual codes (since they have rate $1 - R > 1/2$). Let $\mathcal{A}_{\text{PECK}}$ be a solver for $\text{PECK}_{q,n,k}$ and consider an algorithm $\mathcal{A}_{\text{PEP}}$ that works as follows:

- $\mathcal{A}_{\text{PEP}}$ first samples a codeword from $\mathcal{C}$, simulating $\mathcal{D}_{q,n,k}^{\text{PECK}}$. This is repeated until a codeword $\mathbf{c}$ without repeated entries is found;
- then, it calls $\mathcal{A}_{\text{PECK}}$ on input $\{\mathbf{c}, \mathbf{G}, \mathbf{H}'\}$.

Notice that we need to sample a codeword without repeated entries so that, when $\mathcal{A}_{\text{PECK}}$ outputs a permutation $\mathbf{P}$, this corresponds exactly to the solution to $\text{PEP}_{q,n,k}$. We now show that, for q according to Equation (5), the first phase of the algorithm needs to sample only $O(1)$ codewords: we do this by showing that the probability to get a codeword with all distinct entries is asymptotically lower bounded by a constant. Namely, this probability is

$$\prod_{i=1}^{n-1} \left(1 - \frac{i}{q}\right) > \left(1 - \frac{n-1}{q}\right)^{n-1} > \left(1 - \frac{n}{q}\right)^n. \quad (6)$$

Using the lower bound on q Equation (5), we further get

$$\begin{aligned} \left(1 - \frac{n}{q}\right)^n &> \left(1 - \frac{n}{(n/e)^{\frac{1}{1-R}}}\right)^n = \left(1 - \frac{e^{\frac{1}{1-R}}}{n^{\frac{R}{1-R}}}\right)^n \\ &= \left(\left(1 - \frac{e^{\frac{1}{1-R}}}{n^{\frac{R}{1-R}}}\right)^{n^{\frac{R}{1-R}}}\right)^{n \cdot n^{\frac{-R}{1-R}}} \sim e^{-e^{\frac{1}{1-R}} \cdot n^{\frac{1-2R}{1-R}}}. \end{aligned}$$

First, $e^{\frac{1}{1-R}}$ is a constant $\alpha \geq 1$. Now, observe that $\frac{1-2R}{1-R} \leq 0$ for any value of $R \geq 1/2$, hence $n^{\frac{1-2R}{1-R}}$ decays as n^{-a} for some constant $a \geq 0$. Hence, overall, the bound in the previous inequality is in $O\left(e^{-\alpha \cdot n^{-a}}\right) = O(1)$.

Relation with PKP_{q,n,k} Once attacks on PEP_{q,n,k} are ruled out, the problem mostly resembles PKP_{q,n,k}. Indeed, one must find a permutation sending $\mathbf{c}$ to the kernel of $\mathbf{H}'$. Again, when q is large, with large probability this PKP_{q,n,k} instance has no repeated entries hence it is hard to solve. In the cases we are interested in, we have that q will be less than n but not extremely smaller (say, $q \approx \frac{n}{2}$). We devote the next section to study how PKP solvers can be adapted to our interest case.

6.2 Adapting solvers for PKP

We will now adapt the state-of-the-art solvers for PKP_{q,n,k} [17, 21, 22] to the case of PECK_{q,n,k}, exploiting the fact that the considered instance has multiple solutions. We are mainly concerned with the low q regime since, as we have already said, in the large q regime security of PECK comes from the hardness of PEP.

We first recall the main principle of such algorithms: guessing correctly the effect of the permutation on a subset of coordinates containing an information set is enough to retrieve the whole permutation. Indeed, let $\mathbf{c}$ denote the input codeword and $\mathbf{H}$ indicate the input parity-check matrix: remember that we are trying to find a permutation $\mathbf{P} \in S_n$ so that $\mathbf{cPH}^\top = \mathbf{0}$. Let us consider an information set $J \subseteq \{1, \dots, n\}$ for the code whose parity-check matrix is $\mathbf{H}$, and let $J' \subseteq \{1, \dots, n\}$ of size $k + \ell$, with $1 \leq \ell \leq n - k$ such that $J \subseteq J'$. Without loss of generality, let $J = \{1, \dots, k\}$ and $J' = \{1, \dots, k + \ell\}$, and consider $\mathbf{H}$ in *quasi-systematic form*, i.e., such that it has the identity matrix of side $n - k - \ell$ in the bottom-right corner. Let $\tilde{\mathbf{c}} = \mathbf{cP}$, and split both $\mathbf{H}$ and $\tilde{\mathbf{c}}$ according to the positions indexed by J' and $\bar{J}' = \{1, \dots, n\}/J'$, as:

$$\tilde{\mathbf{H}} = (\tilde{\mathbf{H}}_{J'} \tilde{\mathbf{H}}_{\bar{J}'}) = \begin{pmatrix} \tilde{\mathbf{H}}_1 & \mathbf{0} \\ \tilde{\mathbf{H}}_2 & \mathbf{I}_{n-k-\ell} \end{pmatrix} \quad \text{and} \quad \tilde{\mathbf{c}} = (\mathbf{c}_1, \mathbf{c}_2). \quad (7)$$

Then, one can rewrite the $\text{PECK}_{q,n,k}$ instance as follows:

$$\mathbf{c}_1 \tilde{\mathbf{H}}_1^\top = \mathbf{0} \in \mathbb{F}_q^\ell \quad (8)$$

$$\mathbf{c}_1 \tilde{\mathbf{H}}_2^\top + \mathbf{c}_2 = \mathbf{0} \in \mathbb{F}_q^{n-k-\ell} \quad (9)$$

We can obtain $\mathbf{c}_2$ just by computing $-\mathbf{c}_1 \tilde{\mathbf{H}}_2^\top$, and thus we can obtain the full $\mathbf{P}$: hence, one can just focus on determining $\mathbf{c}_1$.

To do this, we employ a collision search based solver. One can choose a partition of the set J' into two disjoint sets J_1 and J_2 such that $|J_1| = \lfloor \frac{k+\ell}{2} \rfloor$ and $|J_2| = \lceil \frac{k+\ell}{2} \rceil$, then the solver enumerates two lists L_1, L_2 , representing the arrangements of $\mathbf{c}$ on $|J_1|$ and $|J_2|$ entries (see Definition 2), respectively. For simplicity, we can consider $J_1 = \{1, \dots, \lfloor \frac{k+\ell}{2} \rfloor\}$ and $J_2 = \{\lfloor \frac{k+\ell}{2} \rfloor, \dots, \lceil \frac{k+\ell}{2} \rceil\}$. As defined in Equation (1) (in Appendix D) we have that $\mathbf{m} = \psi(\mathbf{c})$ is the vectors of the multiplicities of the values of the finite field within $\mathbf{c}$, then we can define the input lists as:

$$L_i = \mathbb{A}_{|J_i|}(\mathbf{m}) \quad \text{for } i = 1, 2$$

where $\mathbb{A}_t(\mathbf{x})$ denotes the set of arrangement of $\mathbf{x}$ on t position, see Equation (2), whose precise computations to get the size of those lists are given in Lemma 1 in (Appendix D). Since there may be multiple solutions, if we construct the lists L_1, L_2 using all the arrangements, then the algorithm will end up finding all of them. Since we are just interested in finding one of these solutions, we can reduce the size of the input lists so that, on average, only one solution is found. In fact, we can just consider a fraction $\alpha \leq 1$ of all possible arrangements, with $\alpha = 1/\sqrt{N_{\text{sol}}}$. This way, we enumerate two reduced lists L_1^α, L_2^α , each with size $|L_i^\alpha| = \alpha \cdot |L_i|$, for $i = 1, 2$. A specific solution will be found if the arrangements are presents in both lists: this happens with probability α^2 . Hence, the average number of solutions the algorithm finds, in the end, is estimated as $\alpha^2 \cdot N_{\text{sol}} = 1$.

Then, the vectors within the two lists are merged on ℓ symbols of the syndrome value, according to (8). Namely, for each pair $(\mathbf{c}_A, \mathbf{c}_B) \in L_1^\alpha \times L_2^\alpha$ such that $\mathbf{c}_A \mathbf{H}_{J_1}^\top = \mathbf{0} - \mathbf{c}_B \mathbf{H}_{J_2}^\top$, the algorithm computes $\mathbf{c}_2 = -(\mathbf{c}_A, \mathbf{c}_B) \tilde{\mathbf{H}}_2^\top$ and checks if the entries multiset of $(\mathbf{c}_A, \mathbf{c}_B, \mathbf{c}_2)$ is the same as that of $\mathbf{c}$.

In the following lemma we provide an estimation of the cost of finding a solution for a random instance of PECK .

Lemma 2. *Let us consider two codes $\mathcal{C}, \mathcal{C}'$ having generator and parity check matrices $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ and $\mathbf{H}' \in \mathbb{F}_q^{(n-k) \times n}$ respectively. For a given $\mathbf{u} \in \mathbb{F}_q^k$ the collision-search based solver we have described finds a permutation $\mathbf{P} \in S_n$ such that $\mathbf{u} \mathbf{G} \mathbf{P} \mathbf{H}^\top = \mathbf{0}$ with time complexity*

$$T_{\text{Solver}}(\mathbf{u}, \mathbf{G}, \mathbf{H}) = |L_1^\alpha| + |L_2^\alpha| + \frac{|L_1^\alpha| \cdot |L_2^\alpha|}{q^\ell},$$

where $\mathbf{m} = \psi(\mathbf{u} \mathbf{G})$, $|L_1^\alpha| = \left| \mathbb{A}_{\lfloor \frac{k+\ell}{2} \rfloor}(\mathbf{m}) / \sqrt{N_{\text{sol}}} \right|$ and $|L_2^\alpha| = \left| \mathbb{A}_{\lceil \frac{k+\ell}{2} \rceil}(\mathbf{m}) / \sqrt{N_{\text{sol}}} \right|$.

Proof. The solver performs two main steps: the enumeration of the vectors within L_1^α and L_2^α and the subsequent collisions search. Enumerating a list of vectors has a cost equal to its dimension. This explains the first two terms. On average, the number of pairs $(\mathbf{c}_A, \mathbf{c}_B) \in L_1^\alpha \times L_2^\alpha$ that satisfy the condition on ℓ symbols of the syndrome is equal to $|L_1^\alpha| \cdot |L_2^\alpha| \cdot q^{-\ell}$, which corresponds to the number of checks the solver must perform before finding a valid solution, on average. $\square$

Remark 6. The optimal value for ℓ is the one leading to *amortized* lists, i.e., when the number of collisions equals the size of the initial lists.

Remark 7. Notice that if $\mathbf{c}$ does not have repeated entries and the number of solutions is close to 1, the above complexity corresponds to the one already given in [17, 22].

Bounding the cost of the attack Notice that an attacker can repeatedly sample from $\mathcal{D}_{q,n,k}^{\text{PECK}}$ until a codeword $\mathbf{c}$ with some property is obtained. Indeed, there may be certain codewords that make the cost Lemma 2 rather small. This would lead to an overall time complexity $T = T_{\text{Sample}} + T_{\text{Solver}}$, where T_{Sample} is the average number of codewords one has to sample, before a valid $\mathbf{c}$ is found. Among all possible codewords, we will obviously consider only those for which $T_{\text{Sample}} < 2^{128}$. Let d indicate the number of non-zero entries in $\mathbf{m} = \psi(\mathbf{c})$: as we prove in Appendix C, whenever $d \leq 63$, $T_{\text{Sample}} > 2^{128}$. Hence, it must be $d > 63$.

From now on, we neglect T_{Sample} and aim to derive a lower bound for T_{Solver} . For the sake of simplicity, we don't distinguish between sizes of L_1^α and L_2^α and simply write $L = |\mathbb{A}_{\frac{k+\ell}{2}}(\mathbf{m})|$ for the lists size. Since lists shall be amortized, the number of collisions is $L^2/q^\ell = L$, hence (for the sake of readability we defined $v = \frac{k+\ell}{2}$):

$$T_{\text{Solver}} \approx \frac{\mathbb{A}_v(\mathbf{m})}{\sqrt{\frac{\mathbb{A}_n(\mathbf{m})}{q^{n-k}}}} = \frac{q^{\frac{n-k}{2}} \cdot \mathbb{A}_v(\mathbf{m})}{\sqrt{\mathbb{A}_n(\mathbf{m})}}.$$

From Theorem 1 in Appendix D, it follows that:

$$\mathbb{A}_n(\mathbf{m}) \leq \mathbb{A}_{2v}(\mathbf{m}) \cdot \mathbb{A}_{n-2v}(\mathbf{m}^*),$$

which yields the following bound:

$$T \geq \frac{q^{\frac{n-k}{2}}}{\sqrt{\mathbb{A}_{n-2v}(\mathbf{m}^*)}} \geq \frac{q^{\frac{n-k}{2}}}{\sqrt{\mathbb{A}_{n-k-\tilde{\ell}}(\mathbf{m}^*)}}. \quad (10)$$

To estimate ℓ , we consider the case of amortised lists, i.e. it must be $q^\ell = L$, which implies

$$\ell = \frac{1}{2} \cdot (n - k) + \log_q \left(\frac{\mathbb{A}_v(\mathbf{m})}{\sqrt{\mathbb{A}_n(\mathbf{m})}} \right).$$

Using Equation (10) and knowing that for each $\mathbf{m}$, $\mathbb{A}_{t'}(\mathbf{m})$ increases as ℓ gets lower, we get a lower bound on ℓ by choosing $\mathbf{m}^*$ as the multiplicities vector having $n - k$ values equal to 1 and the remaining set to 0:

$$\tilde{\ell} = \underbrace{\frac{1}{2} \cdot (n - k)}_{\approx 63} - \underbrace{\frac{1}{2} \cdot \log_q (\mathbb{A}_{n-k}(\mathbf{m}^*))}_{\approx 50} \geq 12.8$$

We floor this value and consequently obtain $\tilde{\ell} = 12$. Now, we plug in this value in Equation (10) and compute it using, as $\mathbf{m}^*$, the vector having exactly $n - k - \tilde{\ell}$ values equal to 1 and the remaining values equal to 0. With this value, the right hand term of Equation (10) is 2^{130} .

7 Conclusions

We have introduced a new hardness assumption and used it to build a new code-based signature scheme. In a nutshell, the idea is to modify (a permutation version of) LESS so that the secret permutation is used as a trapdoor to map random codewords from the first code into the second one. The main idea is that by avoiding to send the full code, there is no need for Gaussian elimination, which is the major computational bottleneck of LESS. Indeed, our protocol, in its extreme version, has a verification time that is between 50 and 250 times faster than LESS, at the price of only a 2 – 4 times bigger signature size. At the same time, our protocol takes heavy inspiration from the Permuted Kernel Problem and PERK, but it is able to keep a simpler structure, similar to classical Sigma protocols. This allows us to have a slightly faster protocol than PERK, while also having a smaller signature size. Unfortunately, as of now we are able to provide an optimised implementation only for the Low q regime, since it shares a lot of similarities with LESS. Providing a detailed implementation and benchmarking also for the High q regime will be the subject of future work.

We also provide an analysis of the hardness of our assumption, showing that it is resistant against the most commonly used solvers for PEP and PKP. However, a deeper analysis is needed, in particular it would be nice to check whether PECK is really easier than PEP or a reduction between the two can be made.

Finally, as possible direction for future research, we note that the starting code is never used directly in the protocol, besides providing the pool of codewords from which the signer samples. In particular, this suggests that it could be possible to switch from having an instance of PEP as public key to having an instance of the Subcode Equivalence Problem, which is NP-Hard (contrary to Permutation Code Equivalence). This, in theory, could allow for better parameters, since the problem underlying the public key derivation is harder. However, a more careful analysis is needed, in particular about how a smaller code affects sampling for the commitment step.

References

- [1] N. Aaraj et al. *PERK*. Tech. rep. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>. National Institute of Standards and Technology, 2024.
- [2] M. Abdalla, J. H. An, M. Bellare, and C. Namprempre. “From Identification to Signatures via the Fiat-Shamir Transform: Minimizing Assumptions for Security and Forward-Security”. In: *EUROCRYPT 2002*. Ed. by L. R. Knudsen. Vol. 2332. LNCS. Springer, Berlin, Heidelberg, 2002, pp. 418–433. DOI: 10.1007/3-540-46035-7_28.
- [3] G. Alagic, M. Bros, P. Ciadoux, D. Cooper, Q. Dang, T. Dang, J. Kelsey, J. Lichtinger, Y.-K. Liu, C. Miller, et al. “Status Report on the First Round of the Additional Digital Signature Schemes for the NIST Post-Quantum Cryptography Standardization Process”. In: *NIST IR 8528* (2024).
- [4] M. R. Albrecht, B. Benčina, and R. W. F. Lai. “Hollow LWE: A New Spin”. In: *Advances in Cryptology – EUROCRYPT 2025*. Ed. by S. Fehr and P.-A. Fouque. Cham: Springer Nature Switzerland, 2025, pp. 363–392.
- [5] M. Baldi et al. *LESS — Linear Equivalence Signature Scheme*. Tech. rep. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>. National Institute of Standards and Technology, 2024.
- [6] M. Bardet, A. Otmani, and M. Saeed-Taha. “Permutation code equivalence is not harder than graph isomorphism when hulls are trivial”. In: *2019 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2019, pp. 2464–2468.
- [7] A. Barengi, J.-F. Biasse, E. Persichetti, and P. Santini. “LESS-FM: fine-tuning signatures from the code equivalence problem”. In: *Post-Quantum Cryptography: 12th International Workshop, PQCrypto 2021, Daejeon, South Korea, July 20–22, 2021, Proceedings 12*. Springer, 2021, pp. 23–43.
- [8] A. Barengi, J.-F. Biasse, E. Persichetti, and P. Santini. “On the computational hardness of the code equivalence problem in cryptography”. In: *Advances in Mathematics of Communications* 17.1 (2023), pp. 23–55.
- [9] W. Beullens. “Not enough LESS: an improved algorithm for solving code equivalence problems over F_q ”. In: *International Conference on Selected Areas in Cryptography*. Springer, 2020, pp. 387–403.
- [10] W. Beullens, J.-C. Faugère, E. Koussa, G. Macario-Rat, J. Patarin, and L. Perret. “PKP-Based Signature Scheme”. In: *INDOCRYPT 2019*. Ed. by F. Hao, S. Ruj, and S. Sen Gupta. Vol. 11898. LNCS. Springer, Cham, Dec. 2019, pp. 3–22. DOI: 10.1007/978-3-030-35423-7_1.
- [11] J.-F. Biasse, G. Micheli, E. Persichetti, and P. Santini. “LESS is More: Code-Based Signatures Without Syndromes”. In: *AFRICACRYPT 20*. Ed. by A. Nitaj and A. M. Youssef. Vol. 12174. LNCS. Springer, Cham, July 2020, pp. 45–65. DOI: 10.1007/978-3-030-51938-4_3.

- [12] G. Borin, E. Persichetti, P. Santini, F. Pintore, and K. Reijnders. “A guide to the design of digital signatures based on cryptographic group actions”. In: *Cryptology ePrint Archive* (2023).
- [13] A. Chailloux and S. Etinski. “On the (In) security of optimized Stern-like signature schemes”. In: *Designs, Codes and Cryptography* 92.3 (2024), pp. 803–832.
- [14] T. Chou, E. Persichetti, and P. Santini. “On linear equivalence, canonical forms, and digital signatures”. In: *Designs, Codes and Cryptography* (2025), pp. 1–43.
- [15] A. Fiat and A. Shamir. “How To Prove Yourself: Practical Solutions to Identification and Signature Problems”. In: *Advances in Cryptology — CRYPTO’ 86*. Ed. by A. M. Odlyzko. Berlin, Heidelberg: Springer Berlin Heidelberg, 1987, pp. 186–194.
- [16] A. Fiat and A. Shamir. “How to Prove Yourself: Practical Solutions to Identification and Signature Problems”. In: *CRYPTO’86*. Ed. by A. M. Odlyzko. Vol. 263. LNCS. Springer, Berlin, Heidelberg, Aug. 1987, pp. 186–194. DOI: 10.1007/3-540-47721-7_12.
- [17] E. Koussa, G. Macario-Rat, and J. Patarin. “On the complexity of the Permuted Kernel Problem”. In: *Cryptology ePrint Archive* (2019).
- [18] J. Leon. “Computing automorphism groups of error-correcting codes”. In: *IEEE Transactions on Information Theory* 28.3 (1982), pp. 496–511.
- [19] O. Makhnei, V. Pylypiv, and R. Zatorskii. “m-submultisets and m-permutations of multisets elements”. In: *Carpathian Mathematical Publications* 13 (June 2021), pp. 240–258. DOI: 10.15330/cmp.13.1.240-258.
- [20] E. Persichetti and P. Santini. “A New Formulation of the Linear Equivalence Problem and Shorter LESS Signatures”. In: *ASIACRYPT 2023, Part VII*. Ed. by J. Guo and R. Steinfeld. Vol. 14444. LNCS. Springer, Singapore, Dec. 2023, pp. 351–378. DOI: 10.1007/978-981-99-8739-9_12.
- [21] P. Santini, M. Baldi, and F. Chiaraluce. “A Novel Attack to the Permuted Kernel Problem”. In: *2022 IEEE International Symposium on Information Theory (ISIT)*. 2022, pp. 1441–1446. DOI: 10.1109/ISIT50566.2022.9834867.
- [22] P. Santini, M. Baldi, and F. Chiaraluce. “Computational Hardness of the Permuted Kernel and Subcode Equivalence Problems”. In: *IEEE Trans. Inf. Theor.* 70.3 (Mar. 2024), 2254–2270. DOI: 10.1109/TIT.2023.3323068. URL: <https://doi.org/10.1109/TIT.2023.3323068>.
- [23] N. Sendrier. “Finding the permutation between equivalent linear codes: The support splitting algorithm”. In: *IEEE Transactions on Information Theory* 46.4 (2000), pp. 1193–1203.
- [24] N. Sendrier. “On the dimension of the hull”. In: *SIAM Journal on Discrete Mathematics* 10.2 (1997), pp. 282–293.

A Useful concepts about Interactive Proofs and Sigma Protocol

Definition 5 (Binary relation). A binary relation is a finite set $R \subseteq X \times Y$, where $X, Y \subseteq \{0, 1\}^*$. Given $(x, y) \in R$, we say that y is a witness for the statement x . The set $L_R = \{x \in X \mid \exists y \in Y \text{ s.t. } (x, y) \in R\}$ is called the set of true statements for R , or its language.

For a binary relation R we can (informally) define the experiment of Experiment 2 against an adversary $\mathcal{A}$.⁵

Experiment 2: $\text{Exp}_{R, \mathcal{A}}^{\text{H-REL}}(\lambda)$

```

1:  $(x, y) \xleftarrow{\$} R$ 
2:  $y' \xleftarrow{\$} \mathcal{A}(x)$ 
3: return  $(x, y') \in R$ 

```

Definition 6 (Hard Binary Relation). Let $\mathcal{A}$ be an adversary playing the hard-relation experiment $\text{Exp}_{R, \mathcal{A}}^{\text{H-REL}}(\lambda)$ (Experiment 2) against a binary relation R . We define the advantage of $\mathcal{A}$ in the experiment as $\text{Adv}_{R, \mathcal{A}}^{\text{H-REL}}(\lambda) = \Pr(\text{Exp}_{R, \mathcal{A}}^{\text{H-REL}}(\lambda) = 1)$. We say that R is hard if and only if $\text{Adv}_{R, \mathcal{A}}^{\text{H-REL}}(\lambda)$ is negligible in λ for every probabilistic polynomial-time adversary $\mathcal{A}$.

Definition 7 (Interactive Proof). An interactive proof $(\mathcal{P}, \mathcal{V})$ for a binary relation $R \subseteq X \times Y$ is an interactive protocol between two probabilistic polynomial-time machines $\mathcal{P}$ and $\mathcal{V}$. The prover $\mathcal{P}$ takes as input a pair $(x, y) \in R$ while the verifier $\mathcal{V}$ takes as input x . At the end of the protocol, $\mathcal{V}$ either accepts (outputs 1) or rejects (outputs 0). Furthermore, we say that a transcript, i.e. the set of all messages exchanged in a protocol execution, is accepting (rejecting) if $\mathcal{V}$ accepts (rejects, respectively).

Definition 8 (Public-Coin). An interactive proof $(\mathcal{P}, \mathcal{V})$ is public-coin if all $\mathcal{V}$'s random choices are made public.

In the case of three round protocols, the first message sent by the prover is often called *commitment* (**cmt**), the first message sent by the verifier is called *challenge* (**ch**) and the last message sent by the prover is the *response* (**rsp**).

Definition 9 (Sigma Protocol). A Sigma protocol for a relation $R \subseteq X \times Y$ is a three round public coin interactive protocol that satisfies the property of special soundness, (weak) honest-verifier zero-knowledge and completeness:

⁵ To be more precise, the game should depend on the parameter λ and we should define R as a family of relation.

Experiment 3: $\text{Exp}_{\text{DS}, \mathcal{F}}^{\text{EUF-CMA}}$

<pre> 1: (pk, sk) $\xleftarrow{\\$}$ KeyGen(λ) 2: ST $\leftarrow \emptyset$ 3: ($\mathbf{m}, \sigma$) $\xleftarrow{\\$}$ $\mathcal{F}^{\text{O}, \text{OSign}}(\text{pk})$ 4: if $\mathbf{m} \in \text{ST}$ then 5: return 0 6: return $\text{Vf}(\text{pk}, \mathbf{m}, \sigma)$ </pre>	<pre> OSign($\mathbf{m}$): 1: $\sigma \xleftarrow{\\$}$ Sign(sk, $\mathbf{m}$) 2: ST $\leftarrow \text{ST} \cup \{\mathbf{m}\}$ 3: return σ </pre>
--	---

- **Special Soundness:** there exist a polynomial time algorithm called extractor such that on input a true statement $x \in X$ and two accepting transcript $(\text{cmt}, \text{ch}, \text{rsp})$, $(\text{cmt}, \text{ch}', \text{rsp}')$, it returns a witness y such that $(x, y) \in R$.
- **Honest-Verifier Zero-Knowledge:** there exists a polynomial time algorithm $\mathcal{S}$, called simulator, which on input $x \in X$ and a random challenge ch , outputs an accepting conversation $(\text{cmt}, \text{ch}, \text{rsp})$, with the same probability distribution as conversations between an honest $\mathcal{P}$ and $\mathcal{V}$ on input y .
- **Completeness:** an honest prover is able to convince the verifier with overwhelming probability.

A.1 Digital Signature

Digital Signature Schemes are cryptosystems used to provide *integrity*, *authenticity* and *non-repudiation* to digital data.

Definition 10 (Digital Signature Scheme). A digital signature scheme DS is defined by a tuple of polynomial-time algorithms $\text{DS} = (\text{KeyGen}, \text{Sign}, \text{Vf})$, where the first two are probabilistic and the third is deterministic. In particular, we have:

- $(\text{pk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}(\lambda)$: on input a security parameter λ , it outputs a public key pk and the corresponding secret key sk .
- $\sigma \xleftarrow{\$} \text{Sign}(\text{sk}, \mathbf{m})$: on input a private key sk and a message $\mathbf{m}$, the algorithm outputs a signature σ .
- $1/0 \leftarrow \text{Vf}(\text{pk}, \mathbf{m}, \sigma)$: it takes as input a public key pk , a message $\mathbf{m}$ and a signature σ , and outputs 1 (accept) or 0 (reject).

We required that an honestly generated signature is always verified, i.e. for every security parameter λ and message $\mathbf{m}$,

$$\Pr \left(\text{Vf}(\text{pk}, \mathbf{m}, \sigma) = 1 \mid \begin{array}{l} (\text{pk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}(\lambda) \\ \sigma \xleftarrow{\$} \text{Sign}(\text{sk}, \mathbf{m}) \end{array} \right) = 1.$$

The most standard security property for a digital signature scheme is *unforgeability under chosen-message attack*, defined as follows:

Definition 11 (Existential unforgeability under chosen-message attack).

Let $DS = (\text{KeyGen}, \text{Sign}, \text{Vf})$ be a digital signature scheme, let $\mathcal{O}$ be a random oracle and let $\mathcal{F}$ be a forger. We define the advantage of $\mathcal{F}$ playing the EUF – CMA experiment $\text{Exp}_{DS, \mathcal{F}}^{\text{EUF-CMA}}$ (Experiment 3) against DS in the random oracle model as:

$$\text{Adv}_{DS, \mathcal{F}}^{\text{EUF-CMA}}(\lambda) = \Pr \left(\text{Exp}_{DS, \mathcal{F}}^{\text{EUF-CMA}}(\lambda) = 1 \right).$$

We say that DS is existential unforgeable against chosen-message attacks if $\text{Adv}_{DS, \mathcal{F}}^{\text{EUF-CMA}}(\lambda)$ is negligible in λ for every probabilistic polynomial-time forger $\mathcal{F}$.

Remark 8. The forger’s access to the random oracle $\mathcal{O}$ is not strictly required, and its relevance depends on whether the security of the digital signature is given in the Random Oracle Model (ROM) or in the standard model. Since our construction relies on the Fiat-Shamir transform (see Appendix A.2) we included it in the definition.

A.2 The Fiat-Shamir Transform

Firstly introduced in [16], the Fiat-Shamir transform is a widespread heuristic, used to design digital signature schemes starting from public-coin interactive proofs. Intuitively, the idea is to replace the challenge-communication step with a hash function, evaluated on a suitable input. Formally, we have the following definition:

Definition 12 (Fiat-Shamir Signature). Let λ be a security parameter and let $\Pi = (\mathcal{P}, \mathcal{V})$ be a Sigma Protocol for a hard relation $R \subseteq X \times Y$. Let $\mathcal{H}$ be cryptographic hash functions. The signature scheme obtained from Π by applying the Fiat-Shamir transform is a triple of algorithms $FS(\Pi) = (\text{KeyGen}, \text{Sign}, \text{Vf})$ as detailed in algorithm 1.

B Proof of Theorem 3

Let $\{\mathbf{g}_1, \dots, \mathbf{g}_k\}$, with each $\mathbf{g}_i \in \mathbb{F}_q^n$, such that $\mathcal{A} = \text{Span}(\mathbf{g}_1, \dots, \mathbf{g}_k)$. In other words, the vectors $\mathbf{g}_i$ represent the rows of a generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ for $\mathcal{A}$. Now, we count the number of $k \times n$ matrices that have full rank and generate a space which does not intersect with $\mathcal{A}$. Let us denote the rows of such a matrix, which we refer to as $\mathbf{G}'$, as $\{\mathbf{g}'_1, \dots, \mathbf{g}'_k\}$. The number of “valid” choices for $\mathbf{g}'_1$ is $q^n - q^k$. Indeed, choosing $\mathbf{g}'_1 = \mathbf{0}$ implies that $\mathbf{G}'$ generates a space of dimension $< k$. If instead we choose $\mathbf{g}'_1 \in \mathcal{A} \setminus \{\mathbf{0}\}$, we may not have rank deficiency but, for sure, $\mathcal{A}$ would intersect with $\text{Span}(\mathbf{G}')$. Hence, the number of “wrong” choices for $\mathbf{g}'_1$ is q^k .

We now consider the second row of $\mathbf{G}'$. In this case, the number of vectors to avoid is counted as q^{k+1} , since we must avoid any vector which lives in

$$\mathcal{A} \oplus \text{Span}(\mathbf{g}'_1) = \{\mathbf{a} + \mathbf{b} \mid \mathbf{a} \in \mathcal{A}, \mathbf{b} \in \text{Span}(\mathbf{g}'_1)\}.$$

Algorithm 1 Fiat-Shamir Transformation

KeyGen (λ): 1: $(x, y) \xleftarrow{\$} R$ 2: $sk \leftarrow y$ 3: $pk \leftarrow x$ 4: return (pk, sk)	Vf (pk, m, σ): 1: $ch \leftarrow H(cmt, m)$ 2: return $\mathcal{V}(pk, cmt, rsp)$
--	--

Sign (sk, m): 1: $cmt \leftarrow \mathcal{P}(sk)$ 2: $ch \leftarrow H(cmt, m)$ 3: $rsp \leftarrow \mathcal{P}(sk, cmt, ch)$ 4: return $\sigma \leftarrow (cmt, rsp)$	
--	--

To see this, assume we pick $\mathbf{g}'_2 = \mathbf{a} + \beta \mathbf{g}'_1$ for some β . Then, $\text{Span}(\mathbf{g}'_1, \mathbf{g}'_2)$ intersects the space $\text{Span}(\mathbf{a})$: indeed, $\mathbf{g}'_2 - \beta \mathbf{g}'_1 \in \text{Span}(\mathbf{g}'_1, \dots, \mathbf{g}'_2)$ but $\mathbf{g}'_2 - \beta \mathbf{g}'_1 = \mathbf{a} \in \mathcal{A}$. We iterate the reasoning and get that the number of valid matrices $\mathbf{G}'$ is counted as $\prod_{i=0}^{k-1} q^n - q^{k+i}$. We now divide this quantity by the number of changes of basis, to obtain the number of valid choices for $\mathcal{B}$. Since the number of changes of basis is given by

$$|GL_k(\mathbb{F}_q)| = \prod_{i=0}^{k-1} q^k - q^i,$$

we get that the overall number of choices for $\mathcal{B}$ is

$$\begin{aligned} \frac{\prod_{i=0}^{k-1} q^n - q^{k+i}}{\prod_{i=0}^{k-1} q^k - q^i} &= \prod_{i=0}^{k-1} \frac{q^{k-i}}{q^i} \cdot \frac{q^{n-k-i} - 1}{\prod_{i=0}^{k-1} q^{k-i} - 1} \\ &= q^{k^2} \cdot \prod_{i=0}^{k-1} \frac{q^{n-k-i} - 1}{\prod_{i=0}^{k-1} q^{k-i} - 1} \\ &= q^{k^2} \cdot \left[\begin{matrix} n-k \\ k \end{matrix} \right]_q. \end{aligned}$$

To get the probability in the thesis, we divide this by the number of k -dimensional subcodes in $\mathbb{F}_q^n$, which is given by $\left[\begin{matrix} n \\ k \end{matrix} \right]_q$.

C Sampling codewords with same multiset entries

In this Section we do some considerations which are rather handy to analyse the complexity of solvers for PECK when $q < n$. In particular, we show some probability bounds on the complexity of sampling codewords such that every element of the finite field appears exactly n/q times and on the complexity of sampling two codewords with the same multiset.

Sampling a codeword with a uniform amount of each finite field element On average, we have that the number of codewords such that every element of the finite field appears n/q times is

$$q^{-(n-k)} \cdot \frac{n!}{\left(\frac{n!}{q}\right)^q}.$$

Indeed, $\frac{n!}{\left(\frac{n!}{q}\right)^q}$ counts the number of such vectors, while $q^{-(n-k)}$ estimates the probability that a vector is indeed a codeword. We then divide this value by q^k to estimate the probability that a random codeword is valid, and take the reciprocal of this quantity to get the average number of attempts before one such codeword is obtained. This corresponds to

$$N = \frac{q^k}{q^{-(n-k)} \frac{n!}{\left(\frac{n!}{q}\right)^q}} = \frac{q^n \cdot \left(\frac{n!}{q}\right)^q}{n!}.$$

The above expression can be greatly simplified using Stirling's approximation, yielding

$$N \sim q^n \cdot \frac{\left(\sqrt{2\pi n/q} \cdot \left(\frac{n}{e q}\right)^{n/q}\right)^q}{\sqrt{2\pi n} \left(\frac{n}{e}\right)^n} = (2\pi n)^{\frac{q}{2}-1} \cdot q^{-\frac{q}{2}}.$$

Hence, after N attempts, we obtain a valid codeword $\mathbf{c}$. Then, repeating for N times, we obtain the average number of codewords $\mathbf{c}'$ we need to generate.

Sampling Codewords with a Specified Number of Distinct Values We will now analyze the probability to sample a random codeword belonging to a subset of $\mathbb{F}_q^n$, specifically, given a non zero integer $d < q$ we want to define the probability of sampling a codeword with d or less distinct values.

For this purpose we will use the inclusion-exclusion principle. Given n finite sets, this principle provides a formula for the cardinality of the union set, taking into account the overlaps between the sets.

Let us consider $S \subseteq \{0, \dots, q-1\}$ such that $|S| = d < q$, the number of all $\mathbf{v} \in S^n$, i.e. those vectors whose values are all and only those in S is given by:

$$N_{d,n} = \sum_{j=0}^d \binom{d}{j} (d-j)^n$$

This result can be explained using the principle of inclusion-exclusion: in each term of the summation a $0 \leq j \leq d$ is fixed: for all possible ways of choosing j among d distinct values all the vectors containing only those $d-j$ symbols are considered. The coefficient -1^j is a compensating factor so that each vector is counted the correct number of times. In fact, each $\mathbf{v}$ vector with $m \leq d$ distinct

values is counted whenever the j values chosen are among the $d - m$ values that are not within $\mathbf{v}$, so this vector is counted a number of times equal to

$$\sum_{j=0}^{d-m} (-1)^j \binom{d-m}{j}$$

This corresponds to Newton's binomial, which turns out to always sum to 0 except when $m = d$ and is equal to 1. Then, chosen $d \leq q$, we can use this result to obtain that the probability of sampling a codeword with d or less distinct values is given by:

$$P_{n,d} = \frac{\sum_{d=1}^{\bar{d}} \binom{q}{d} N_{d,n}}{q^n} \quad (11)$$

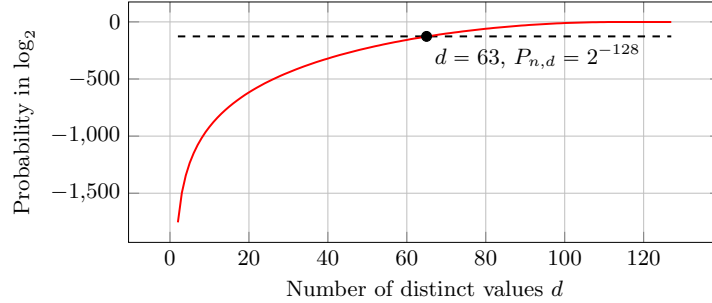


Fig. 8: Probability to sample a codeword with d or less distinct values.

D Arrangements of multiset with repeated entries on a subset of coordinates

We start by showing the proof of Lemma 1.

Proof. The terms of the sum can be explained as follows: for a given $\mathbf{m} = (\tilde{m}_0, \dots, \tilde{m}_{q-1}) \in \mathbb{M}_t(\psi(\mathbf{x}))$, the number of vectors $\mathbf{v} \in \mathbb{X}^t$ such that $\psi(\mathbf{v}) = \mathbf{m}$ can be computed by using the multinomial coefficient, which provides the number of permutation of t elements with $\tilde{m}_0, \dots, \tilde{m}_{q-1}$ repetition. It can be expressed as $\binom{t}{m_0, \dots, m_{q-1}} = \binom{t}{m_0} \binom{n-m_0}{m_1} \dots \binom{t-(m_0+m_1+\dots+m_{q-2})}{m_{q-1}}$. Thus, the overall size of the set is given by each multinomial coefficient for each $\mathbf{m} \in \mathbb{M}_t(\psi(\mathbf{x}))$. $\square$

We will now show an efficient algorithm to compute the size of this set:

For a given $\mathbf{c} \in \mathbb{X}^n$ and $t \leq n$ the algorithm computes the size of the set $\mathbb{A}_t(\psi(\mathbf{x}))$ by following an iterative approach with state: in the i -th iteration the number of all the possible ways of distributing m_i values on $t, t_1, \dots, t_j, \dots, 1$

Algorithm 2

Input: $\mathbf{x} \in \mathbb{X}^n$, $t \leq n$ **Internal Parameters:** $\text{partial_results} \leftarrow [0, \dots, 0] \in \mathbb{N}^t$ **Output:** $|\mathbb{A}_t(\psi(\mathbf{x}))|$

```
 $m := \psi(\mathbf{x})$ 
 $\text{partial\_results}[0] := 1.$ 
for  $m_i$  in  $\tilde{m}$  do
  for  $t_j = t$  to 0 step -1 do
     $nr := 0$ ,  $\text{binom} := 1$ 
    for  $\tilde{m}_k = 0$  to  $\min(t_j, m_i)$  do
       $nr := nr + \text{binom} \cdot \text{partial\_results}[t_j - \tilde{m}_k]$ 
       $\text{binom} := \text{binom} \cdot \frac{t_j - \tilde{m}_k}{\tilde{m}_k + 1}$ 
     $\text{partial\_res}[t_j] := nr$ 
return  $\text{partial\_res}[t]$ 
```

coordinates, with $t_j = t - j$, is computed. Specifically, given a t_j subset of coordinates, the algorithm computes the number of possible ways to arrange $\tilde{m}_k$ values, for each $0 \leq \tilde{m}_k \leq \min(m_i, t_j)$, on the t_j entries taking into account the contribution of the $m_0, \dots, m_{i-1}$ other multiplicities on the remaining $t_j - \tilde{m}_k$ entries.

We now prove Theorem 1. As a warm up, we give a theorem which represents a special case of a more general result we will provide later on.

Theorem 4. *Let $\mathbf{m} = (m_0, \dots, m_{q-1})$ with $\text{wt}(\mathbf{m}) = d$. Let $\text{sum}(\mathbf{m}) = n$: then, for any $t \leq n$, we have*

$$d \cdot \mathbb{A}_{t-1}(\mathbf{m}^*) \geq \mathbb{A}_t(\mathbf{m}) \geq d \cdot \mathbb{A}_{t-1}(\overline{\mathbf{m}}),$$

where $\overline{\mathbf{m}}, \mathbf{m}^* \in \mathbb{E}_1(\mathbf{m})$ are such that

$$\forall \mathbf{m}' \in \mathbb{E}_1(\mathbf{m}), \quad \mathbf{m}' \neq \mathbf{m}^* : \quad \mathbb{A}_{t-1}(\mathbf{m}') \leq \mathbb{A}_{t-1}(\mathbf{m}^*),$$

$$\forall \mathbf{m}' \in \mathbb{E}_1(\mathbf{m}), \quad \mathbf{m}' \neq \overline{\mathbf{m}} : \quad \mathbb{A}_{t-1}(\mathbf{m}') \geq \mathbb{A}_{t-1}(\overline{\mathbf{m}}).$$

Proof. The proof is based on a simple counting argument. Indeed, the arrangements on t positions can be enumerated by 1) picking one element, 2) updating the vector of multiplicities, 3) enumerating all arrangements on $t - 1$ positions, using the new vector of multiplicities. Since we are considering a vector $\mathbf{m}$ with weight d , for the first element we have only d choices. Assume the first element we pick has value equal to z : then, the vector of multiplicities is obtained by subtracting 1 from the z -th element of $\mathbf{m}$. Let us denote such a vector by $\mathbf{1}^{(z)}$, i.e.,

$$\mathbf{1}^{(z)} = (\underbrace{0, \dots, 0}_{\substack{z-1 \\ \text{positions}}}, 1, \underbrace{0, \dots, 0}_{\substack{q-z \\ \text{positions}}})$$

Hence, we have:

$$\mathbb{A}_t(\mathbf{m}) = \sum_{\substack{z \in \{0, \dots, q-1\} \\ m_z \geq 1}} \mathbb{A}_{t-1}(\mathbf{m} - \mathbf{1}^{(z)}).$$

We have d valid indices z , hence the above sum contains d terms. By definition, $\mathbb{A}_{t-1}(\overline{\mathbf{m}})$ is a lower bound for all the terms in the sum. Analogously, we prove the upper bound. $\square$

We are now ready to consider the general case.

First, we observe that $d = \mathbb{A}_1(\mathbf{m})$, hence re-write the previous bound as

$$\mathbb{A}_1(\mathbf{m}) \cdot \mathbb{A}_{t-1}(\mathbf{m}^*) \geq \mathbb{A}_t(\mathbf{m}) \geq \mathbb{A}_1(\mathbf{m}) \cdot \mathbb{A}_{t-1}(\overline{\mathbf{m}}).$$

Now, replace 1 with any $u \leq t$: the general form of the bound we were looking for is obtained. Formally

Proof (Theorem 1). For any value of u , we can express $\mathbb{A}_t(\mathbf{m})$ as

$$\mathbb{A}_t(\mathbf{m}) = \sum_{\mathbf{a} \in \mathbb{A}_u(\mathbf{m})} \mathbb{A}_{t-u}(\mathbf{m} - \psi(\mathbf{a})).$$

The above sum contains $\mathbb{A}_u(\mathbf{m})$ terms and can be easily lower bounded by taking the minimum of the addends. Now: $\psi(\mathbf{a})$ is such that $\text{sum}(\psi(\mathbf{a})) = u$ and obviously $\mathbf{m} - \psi(\mathbf{a}) \in \mathbb{N}^q$, i.e., every coordinates is ≥ 0 . So, the set of all the valid $\mathbf{m} - \psi(\mathbf{a})$ is exactly $\mathbb{E}_t(\mathbf{m})$. This proves the lower bound thesis; with analogous reasoning, we can prove the upper bound. $\square$